Отчёт по лабораторной работе №2

Имитационное моделирование

Петрова Алевтина Александровна

Содержание

1. Задание

- Создать рабочий каталог для всего курса.
- Создать рабочее пространство для программ в рамках лабораторной работы.
- Выполнить все задания по тексту лабораторной работы.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
    - чистый код;
    - jupyter notebook;
    - документацию в формате Quarto.
    - Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
    - чистый код;
    - jupyter notebook;
    - документацию в формате Quarto.
    - Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

2. Теоретическое введение

2.0.1 Модель SIR

Модель SIR есть классическая и фундаментальная математическая модель эпидемиологии, описывающая распространение инфекционного заболевания в закрытой популяции.

2.0.1.1 Базовая модель

Модель SIR делит всю популяцию на три взаимосвязанные группы (компартменты), что отражено в её названии:

S — Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться. I — Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию. R — Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в процессе передачи.

Основная цель модели: не предсказать судьбу конкретного человека, а понять общую динамику эпидемии — будет ли она разрастаться, как быстро, сколько людей в итоге переболеет, как влияют карантинные меры.

2.0.1.2 Трёхпараметрическая модель SIR

В классической двухпараметрической модели (β, γ) параметр β (коэффициент заражения) является составным. Он скрывает в себе два процесса:

- Контакт между людьми (поведенческий, управляемый фактор).
- Передачу инфекции при контакте (биологический фактор).

Трёхпараметрическая модель делает это разделение явным:

- c — среднее число контактов (достаточно тесных для передачи инфекции) одного человека в единицу времени.
- β — вероятность передачи инфекции при одном контакте между заразным и восприимчивым (безразмерная величина, от 0 до 1).
- γ — скорость выздоровления (доля инфицированных, выздоравливающих в единицу времени). Как и раньше, 1/γ — средняя продолжительность заразного периода. Итоговый параметр силы заражения теперь выражается как произведение: c * β.

2.0.2 Модель Лотки–Вольтерры

Модель Лотки-Вольтерры — это фундаментальная математическая модель в экологии, описывающая динамику взаимодействия двух видов: хищников и жертв. Она была независимо предложена в 1920-х годах:

- Альфредом Лоткой (1925) для химических реакций.
- Витторио Вольтеррой (1926) для объяснения колебаний улова рыбы в Адриатическом море.

Модель демонстрирует, как даже простая система взаимодействий может порождать сложные колебательные режимы, объясняя циклические изменения численности в природных экосистемах.

3. Цель работы

Цель данной лабораторной работы — изучить основные модели SIR и Лотки–Вольтерры, а так же изучить аспекты их программной реализации.

4. Выполнение лабораторной работы

4.1 Подготовка рабочего пространства

В этом разделе мы будем использовать уже известные нам скрипты из 1ой лабораторной работы.

Создадим каталог проекта DrWatson с помощью скрипта setup_project.jl (рис. 1).

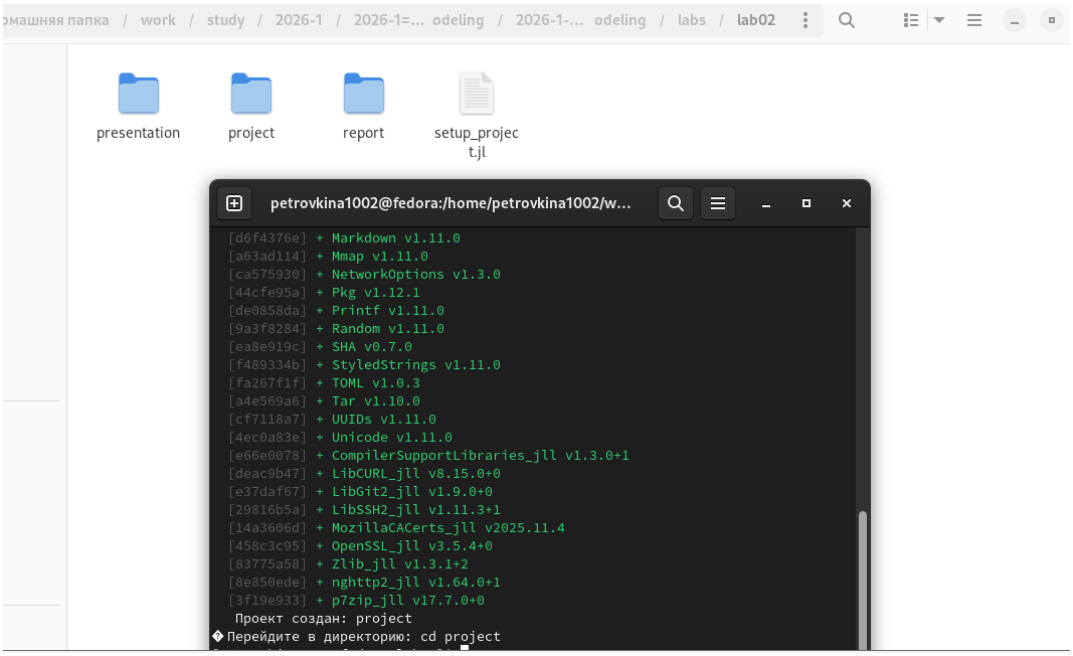


Рисунок 1: Создание каталога проекта DrWatson для лабораторных

Добавим необходимые пакеты с помощью скрипта add_packages.jl (рис. 2).

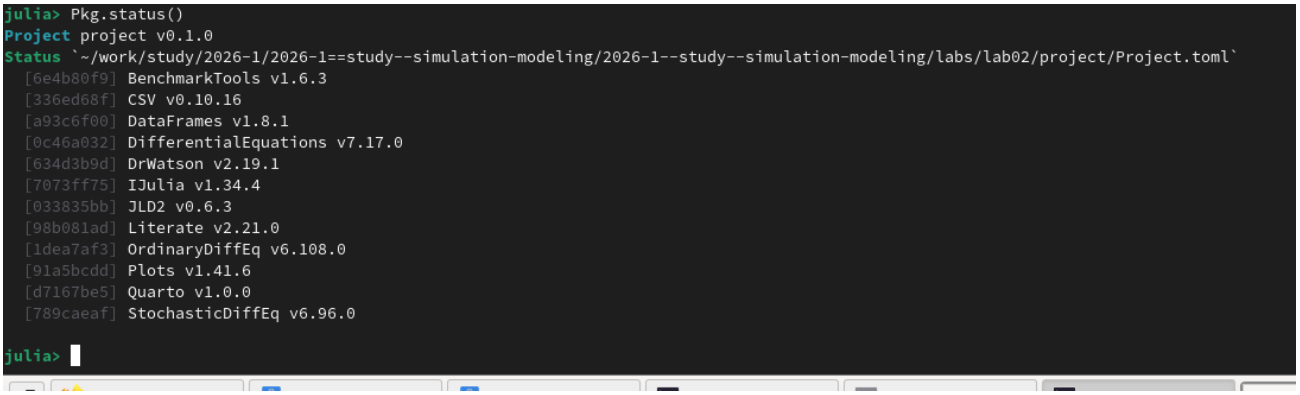


Рисунок 2: Добавление необходимых пакетов

Проверим корректность установки с помощью скрипта scripts/test_setup.jl (рис. 3).

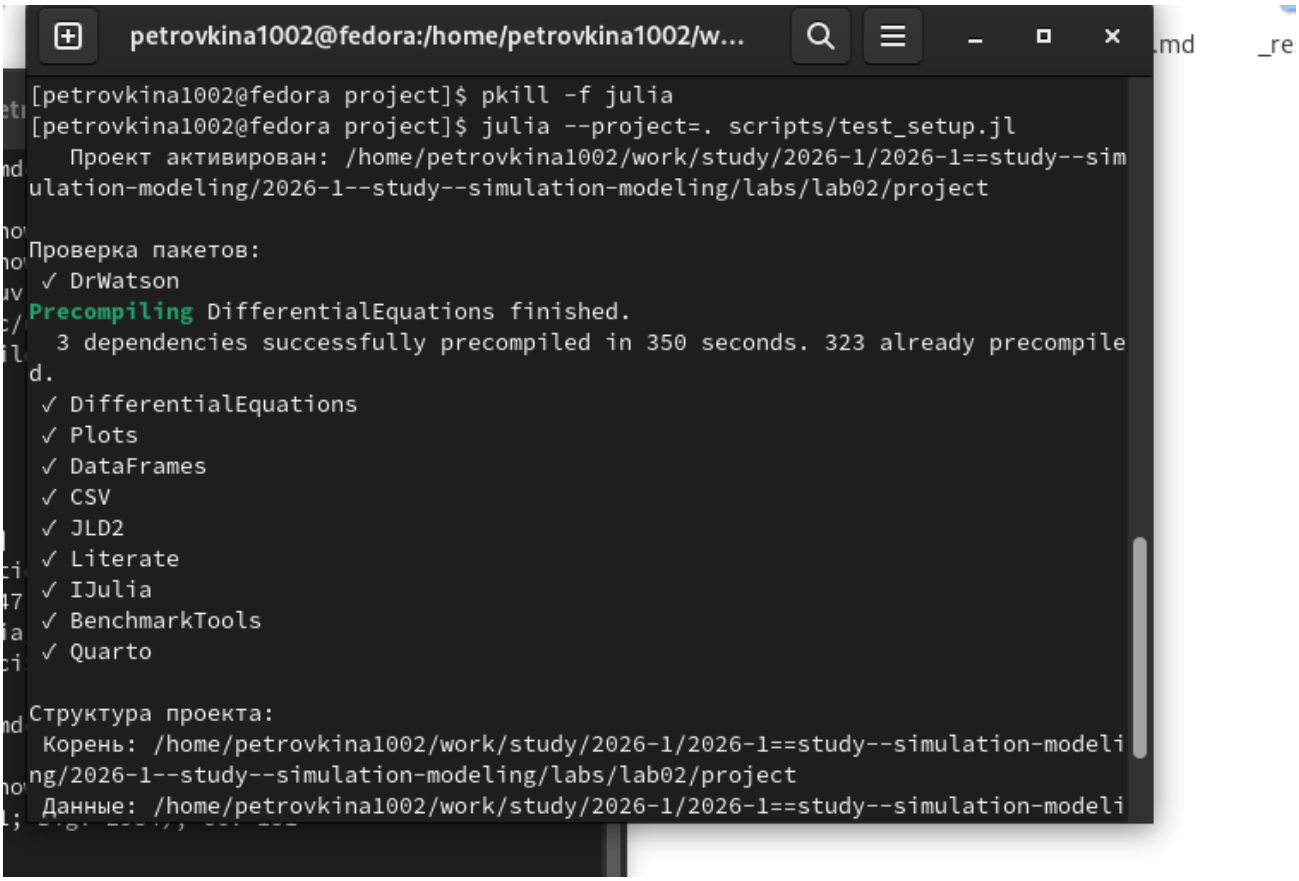


Рисунок 3: Проверка установки

4.2 Основные модели

4.2.1 Модель SIR

Создаём файл scripts/sir_ode.jl с реализацией модели (рис. 4).

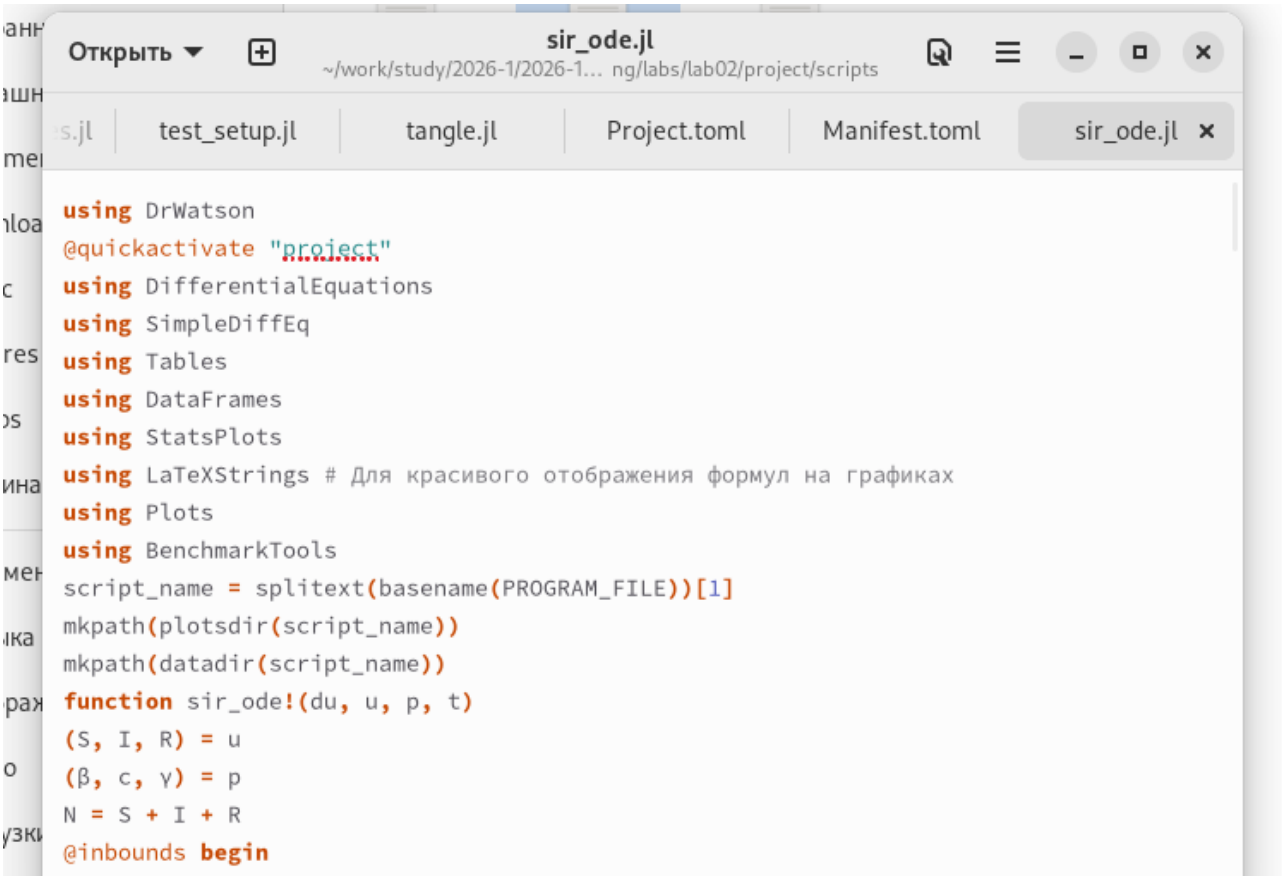


Рисунок 4: scripts/sir_ode.jl

Создадим производные форматы (рис. 5).

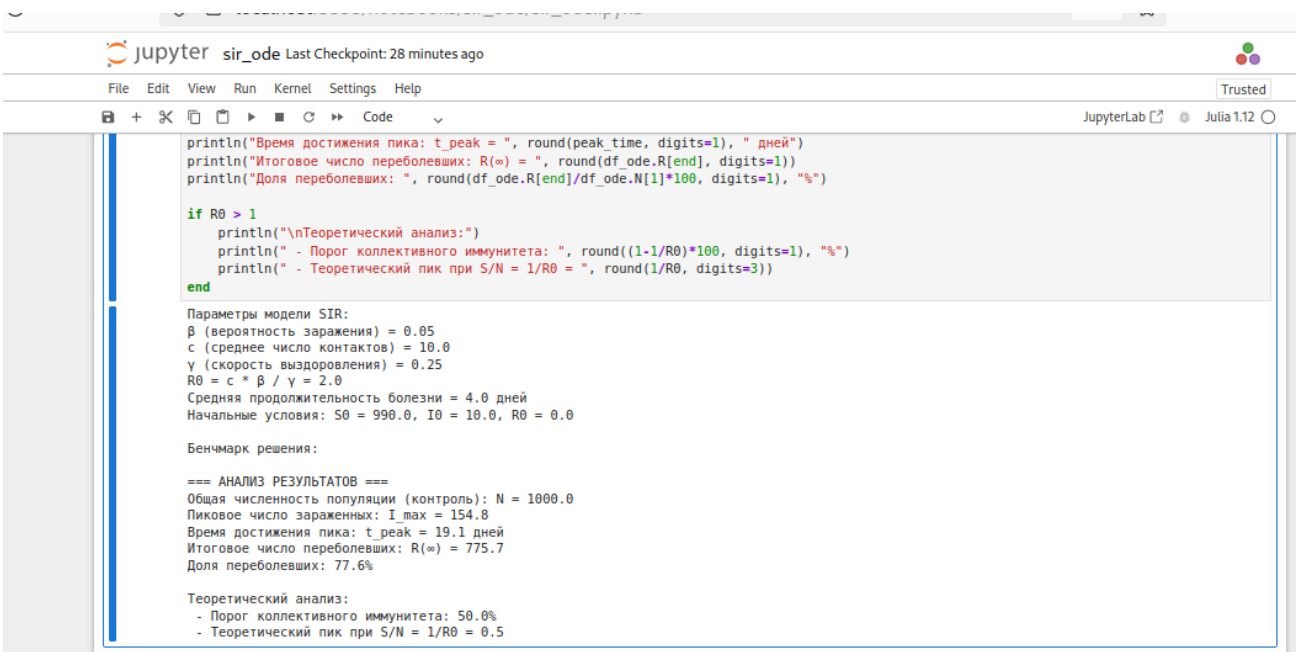


Рисунок 5: scripts/sir_ode.jl

Просмотрим каталог с графиками (рис. 6).

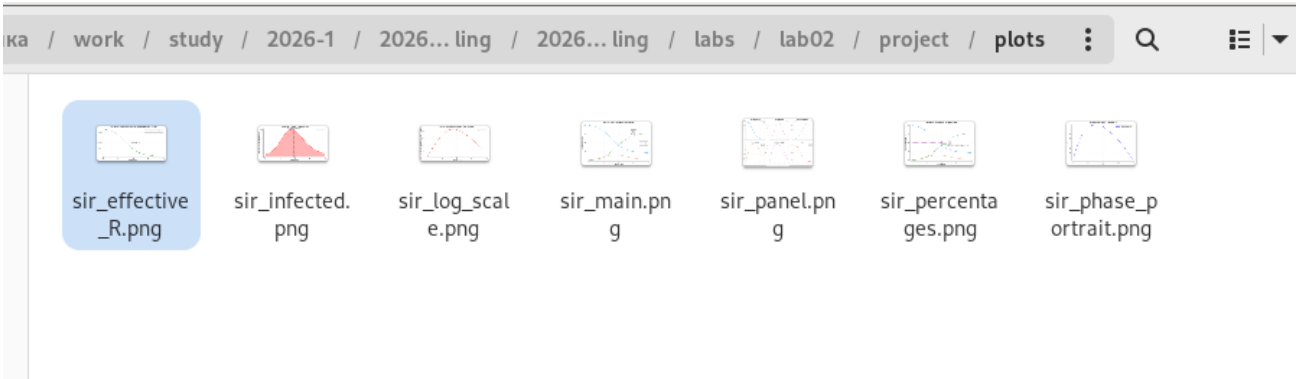


Рисунок 6: Каталог с графиками

Просмотрим qmd файл (рис. 7).

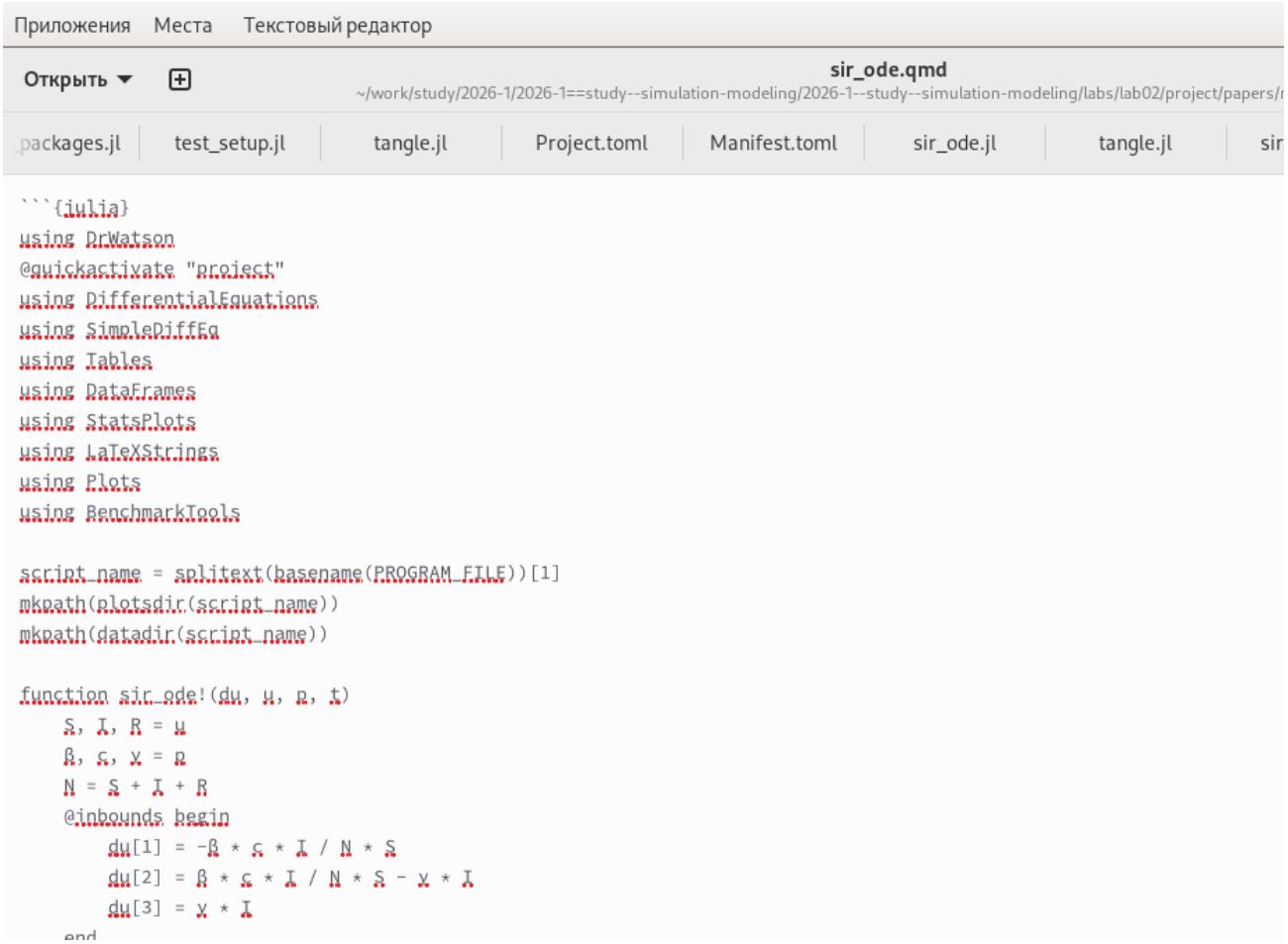


Рисунок 7: qmd файл

Просмотрим jupyter-notebook и запустим все ячейки (рис. 8).

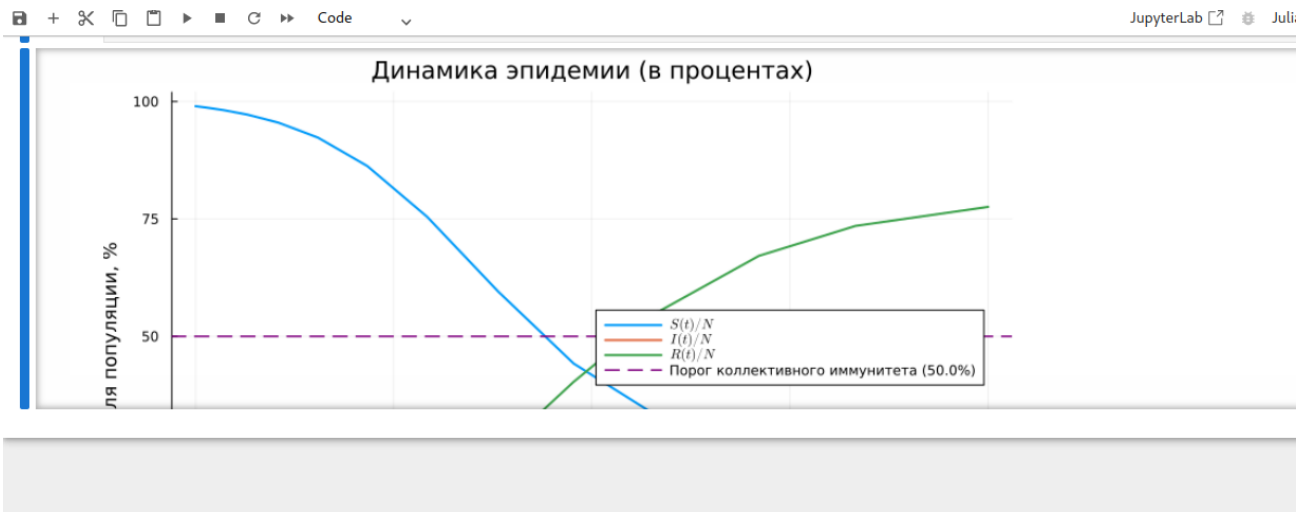


Рисунок 8: Выполнение Jupyter-ноутбука

4.2.2 Модель Лотки–Вольтерры

Создаём файл scripts/lv_ode.jl с реализацией модели (рис. 9).

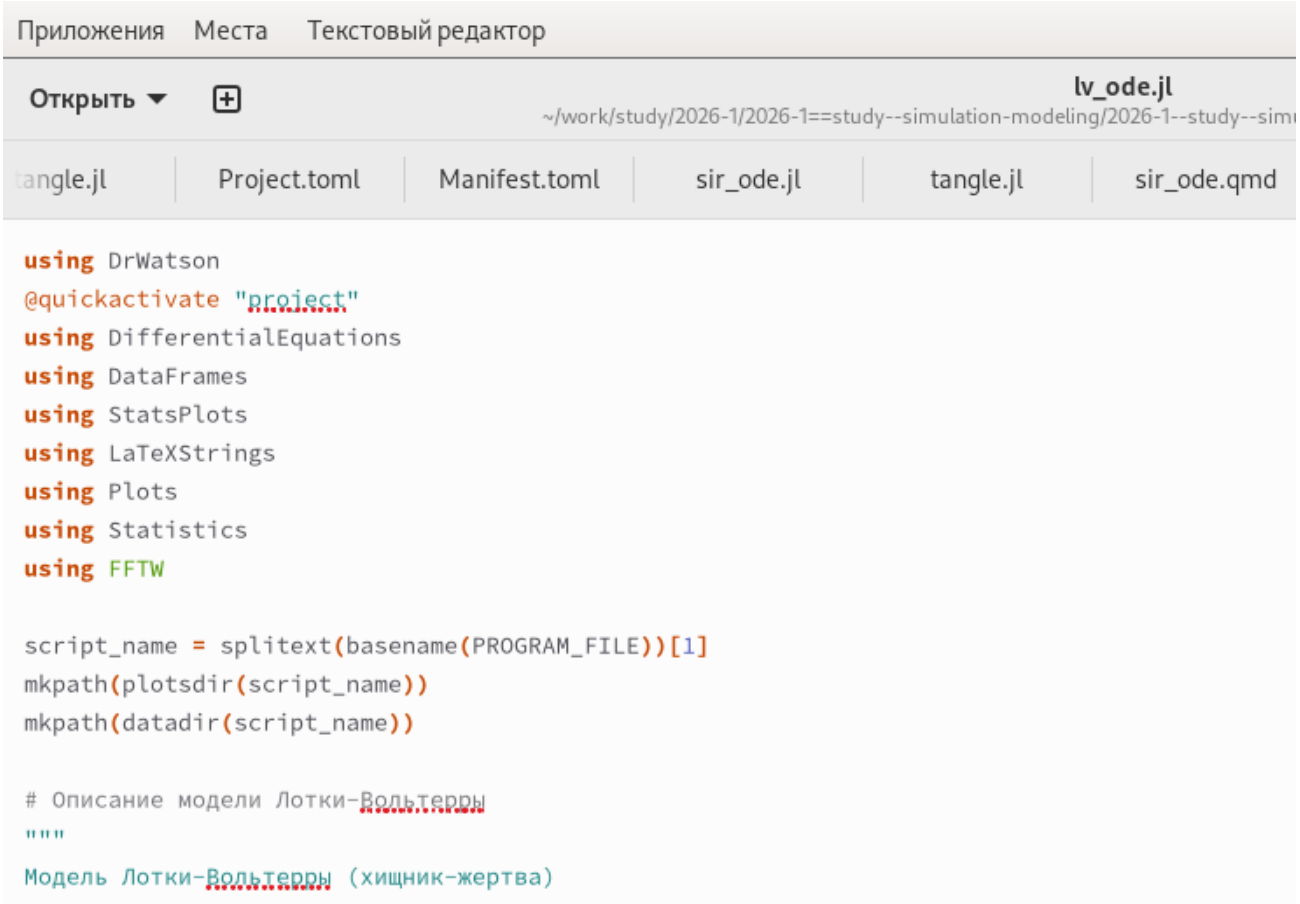


Рисунок 9: scripts/lv_ode.jl

Создадим производные форматы (рис. 10).

```
tion-modeling/2026-1--study--simulation-modeling/labs/lab02/project/scripts/lv_o
de.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modelin
g/2026-1--study--simulation-modeling/labs/lab02/project/markdown/lv_ode/lv_ode.q
md`
 ✓ Quarto: /home/petrovkina1002/work/study/2026-1/2026-1==study--simulation-mode
ling/2026-1--study--simulation-modeling/labs/lab02/project/markdown/lv_ode/lv_od
e.qmd
[ Info: generating notebook from `~/work/study/2026-1/2026-1==study--simulation-
modeling/2026-1--study--simulation-modeling/labs/lab02/project/scripts/lv_ode.jl
`
[ Info: writing result to `~/work/study/2026-1/2026-1==study--simulation-modelin
g/2026-1--study--simulation-modeling/labs/lab02/project/notebooks/lv_ode/lv_ode.
ipynb`
 ✓ Notebook: /home/petrovkina1002/work/study/2026-1/2026-1==study--simulation-mo
deling/2026-1--study--simulation-modeling/labs/lab02/project/notebooks/lv_ode/lv
_ode.ipynb

Готово! Все файлы созданы.
[petrovkina1002@fedora scripts]$
```

Рисунок 10: scripts/lv_ode.jl

Просмотрим каталог с графиками (рис. 11).

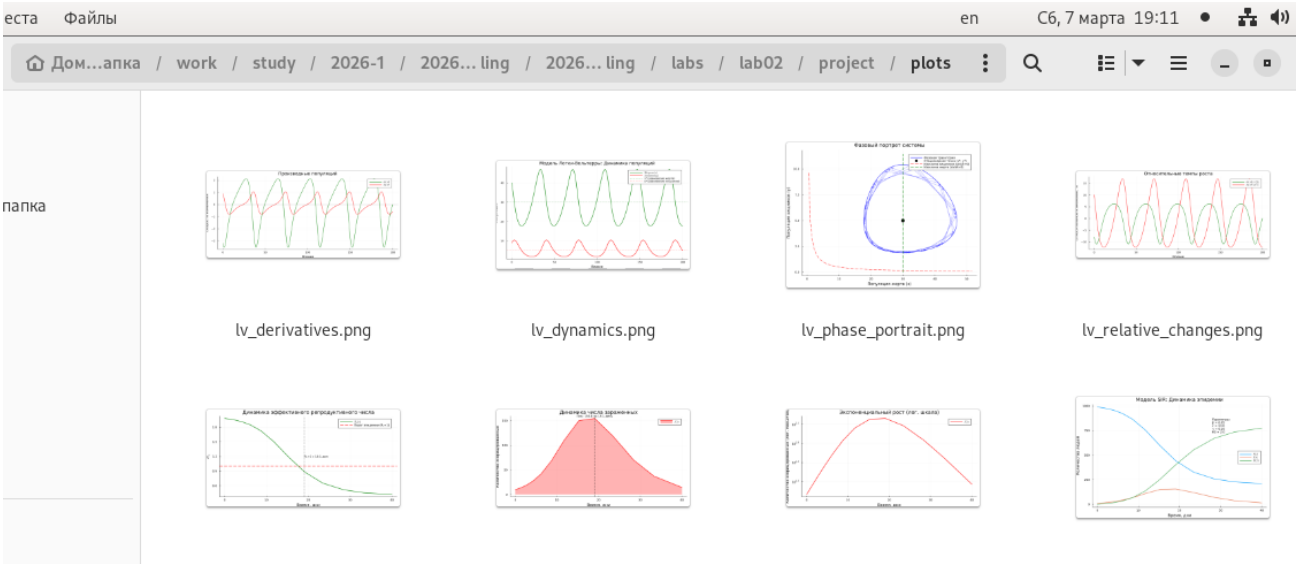


Рисунок 11: Каталог с графиками

Просмотрим qmd файл (рис. 12).

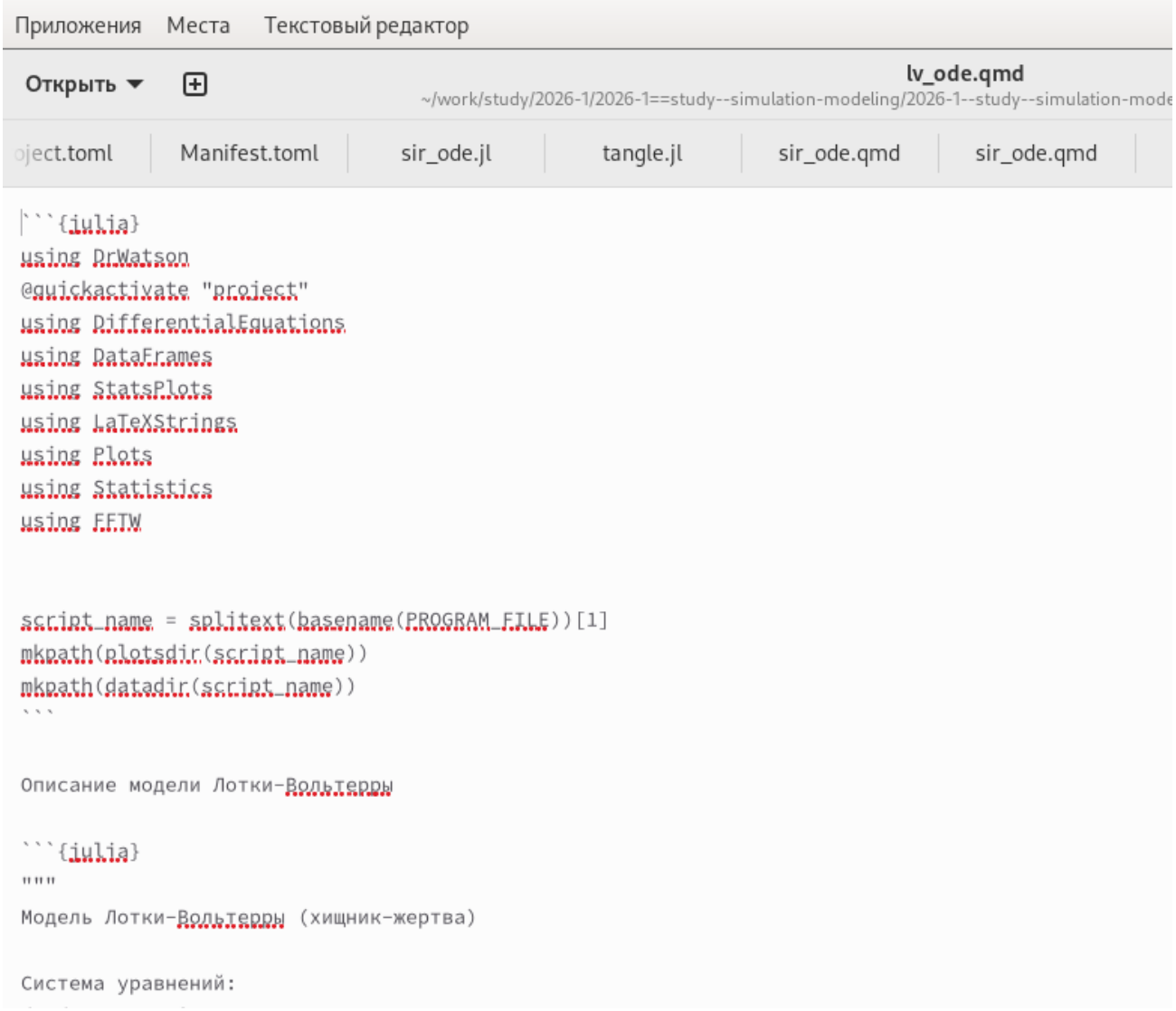


Рисунок 12: qmd файл

Просмотрим jupyter-notebook и запустим все ячейки (рис. 13).

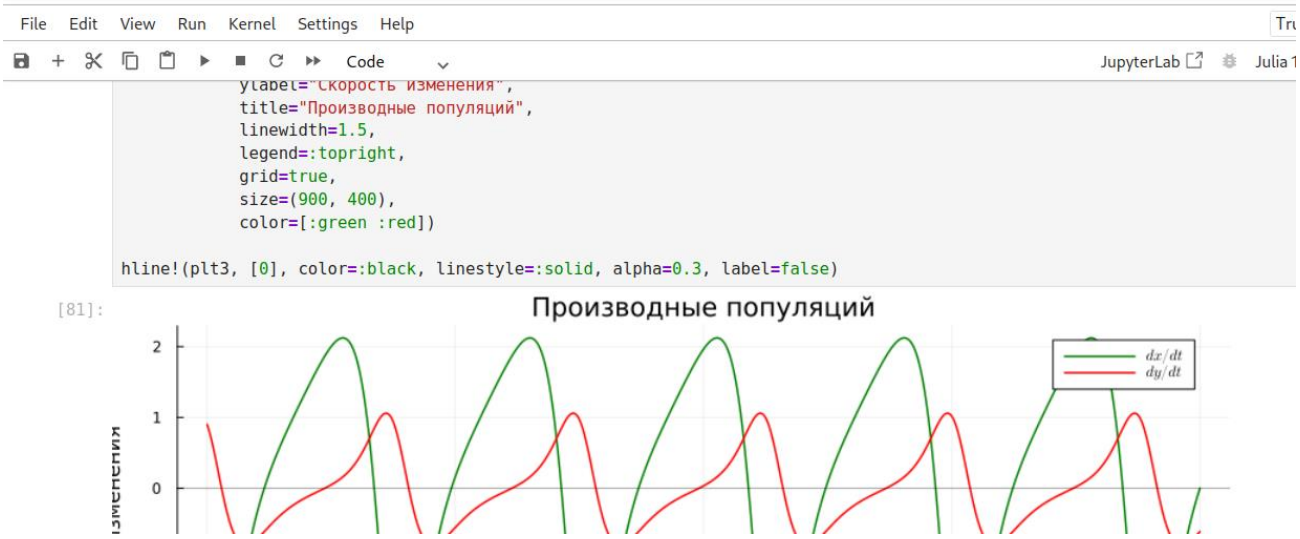


Рисунок 13: Выполнение Jupyter-ноутбука

5. Модель SIR

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate "project"
using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings  # Для красивого отображения формул на графиках
using Plots
default(fmt = :png)
using BenchmarkTools

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (β, c, γ) = p
    N = S + I + R
    @inbounds begin
        du[1] = -β * c * I / N * S
        du[2] = β * c * I / N * S - γ * I
        du[3] = γ * I
```

```
    end
    nothing
end
```

```
  Activating project at `~/work/study/2026-1/2026-1==simulation-modeling/2026-1-
-study--simulation-modeling/labs/lab02/project`
┌ Warning: DrWatson could not find find a project file by recursively checking
given `dir` and its parents. Returning `nothing` instead.
│ (given dir: /home/kali)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84

sir_ode! (generic function with 1 method)
```

Параметры модели

```
δt = 0.1
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0]  # S, I, R
p = [0.05, 10.0, 0.25]   # β, c, γ
```

```
3-element Vector{Float64}:
  0.05
 10.0
  0.25
```

Расчет базового репродуктивного числа

```
R0 = (p[2] * p[1]) / p[3]  # R₀ = (c * β) / γ
```

```
2.0
```

Создание и решение задачи

```
prob_ode = ODEProblem(sir_ode!, u0, tspan, p)
sol_ode = solve(prob_ode, dt = δt)
```

```
retcode: Success
Interpolation: 3rd order Hermite
t: 15-element Vector{Float64}:
  0.0
  0.1
  0.5331242537191838
  1.3919009080055345
  2.61021221798537
  4.181182326044298
  6.180124045228886
  8.664952146237743
 11.691425523605613
 15.27937105275849
 19.0829636767576
 23.423425969211983
 28.429494244539086
 33.29877943214063
```

```
 40.0
u: 15-element Vector{Vector{Float64}}:
 [990.0, 10.0, 0.0]
 [989.4990153200853, 10.247898072276143, 0.2530866076385313]
 [987.1852964117763, 11.391111426852984, 1.4235921613706282]
 [981.8312121172252, 14.026022241732, 4.142765641042832]
 [972.139229760034, 18.757816192500677, 9.102954047465346]
 [954.9905468224117, 27.007704055602183, 18.001749121986144]
 [923.070667251774, 41.929769879700146, 34.99956286852594]
 [862.6705746724687, 68.49345686513698, 68.83596846239446]
 [754.3022914470604, 109.7418645302877, 135.95584402265195]
 [595.101409623216, 150.41363540416995, 254.4849549726141]
 [442.01411154679107, 154.80694904937002, 403.17893940383897]
 [326.895101656965, 119.0685959489173, 554.0363023941178]
 [258.5085195746908, 70.11051759777406, 671.3809628275352]
 [227.58338228204383, 37.328569316974345, 735.0880484009818]
 [209.84121402510556, 14.473644966335414, 775.6851410085591]
```

Подготовка данных в DataFrame

```
df_ode = DataFrame(Tables.table(sol_ode'))
rename!(df_ode, ["S", "I", "R"])
df_ode[!, :t] = sol_ode.t
df_ode[!, :N] = df_ode.S + df_ode.I + df_ode.R  # Общая численность популяции
```

```
15-element Vector{Float64}:
 1000.0
 1000.0
  999.9999999999999
 1000.0
 1000.0
 1000.0
 1000.0000000000001
 1000.0000000000001
 1000.0
 1000.0
 1000.0
 1000.0000000000001
 1000.0
 1000.0
 1000.0
```

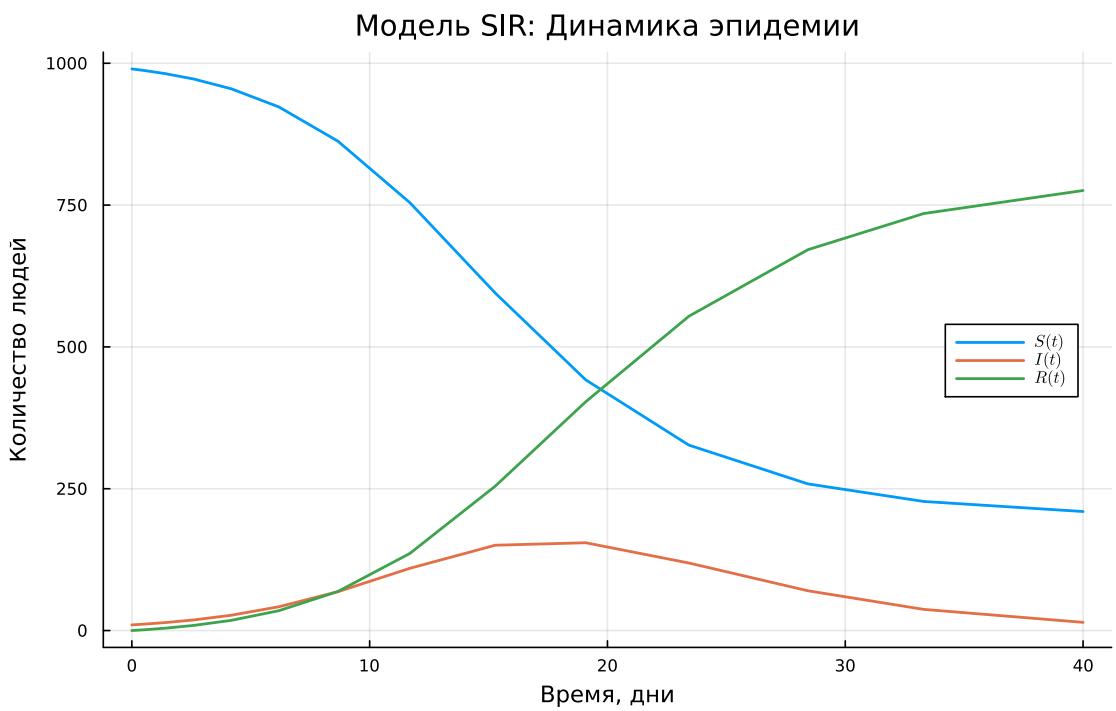
Вывод параметров модели

```
println("Параметры модели SIR:")
println("β (вероятность заражения) = ", p[1])
println("c (среднее число контактов) = ", p[2])
println("γ (скорость выздоровления) = ", p[3])
println("R₀ = c * β / γ = ", round(R0, digits=3))
println("Средняя продолжительность болезни = ", round(1/p[3], digits=2), "
дней")
println("Начальные условия: S₀ = ", u0[1], ", I₀ = ", u0[2], ", R₀ = ", u0[3])
```

```
Параметры модели SIR:
β (вероятность заражения) = 0.05
c (среднее число контактов) = 10.0
γ (скорость выздоровления) = 0.25
R₀ = c * β / γ = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S₀ = 990.0, I₀ = 10.0, R₀ = 0.0
```

1. ОСНОВНОЙ ГРАФИК: динамика всех трех групп

```
plt1 = @df df_ode plot(:t,
                       [:S :I :R],
                       label=[L"S(t)" L"I(t)" L"R(t)"],
                       xlabel="Время, дни",
                       ylabel="Количество людей",
                       title="Модель SIR: Динамика эпидемии",
                       linewidth=2,
                       legend=:right,
                       grid=true,
                       size=(800, 500))
```



Добавление аннотаций с параметрами

```
annotate!(plt1, maximum(df_ode.t) * 0.7, maximum(df_ode.N) * 0.8,
          text("Параметры:\nβ = $(p[1])\nc = $(p[2])\nγ = $(p[3])\nR₀ =
$(round(R0, digits=2))",
               8, :left))
```

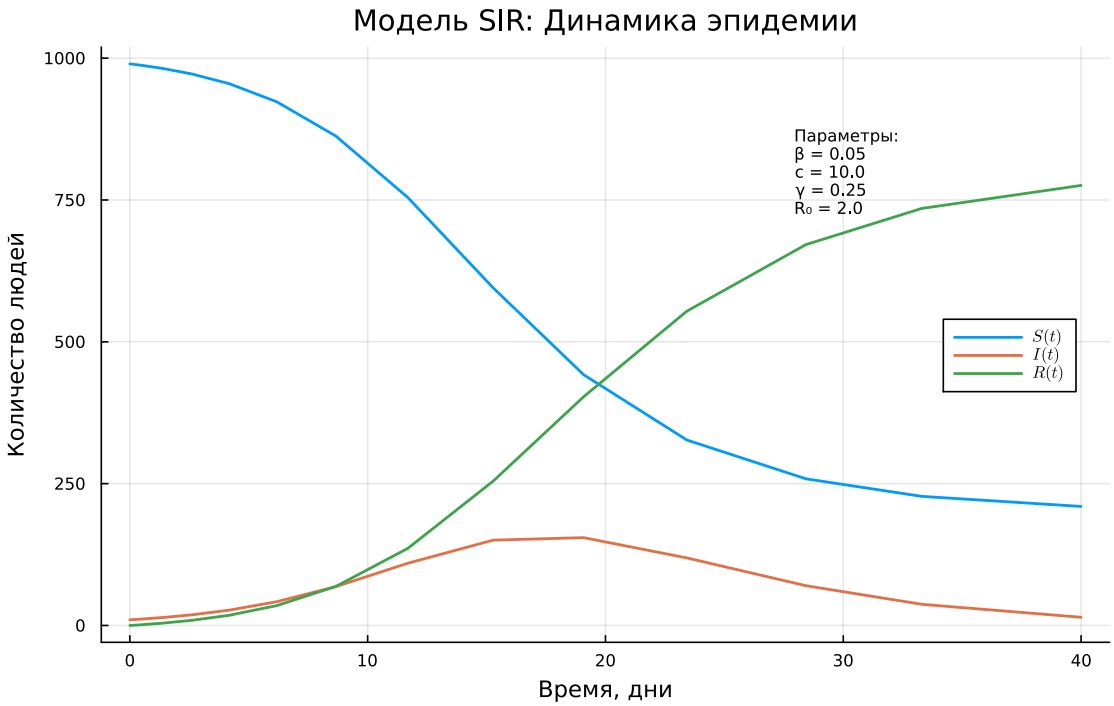
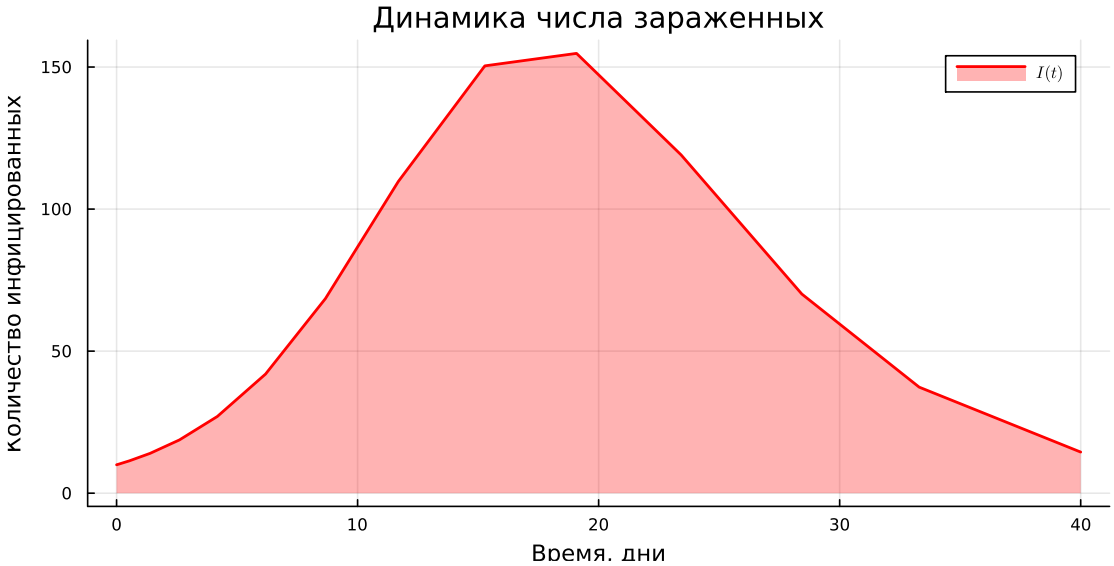


График только инфицированных (I)

```
plt2 = @df df_ode plot(:t, :I,
                       label=L"I(t)",
                       xlabel="Время, дни",
                       ylabel="Количество инфицированных",
                       title="Динамика числа зараженных",
                       color=:red,
                       linewidth=2,
                       fill=(0, 0.3, :red),
                       grid=true,
                       size=(800, 400))
```



Отметка пика эпидемии

```
        peak_idx = argmax(df_ode.I)
        peak_time = df_ode.t[peak_idx]
        peak_value = df_ode.I[peak_idx]
        vline!(plt2, [peak_time], color=:black, linestyle=:dash, label=false,
linewidth=1)
        annotate!(plt2, peak_time, peak_value * 1.05,
                  text("Пик: $(round(peak_value, digits=1)) на
$(round(peak_time, digits=1)) день",
                       8, :top))
```

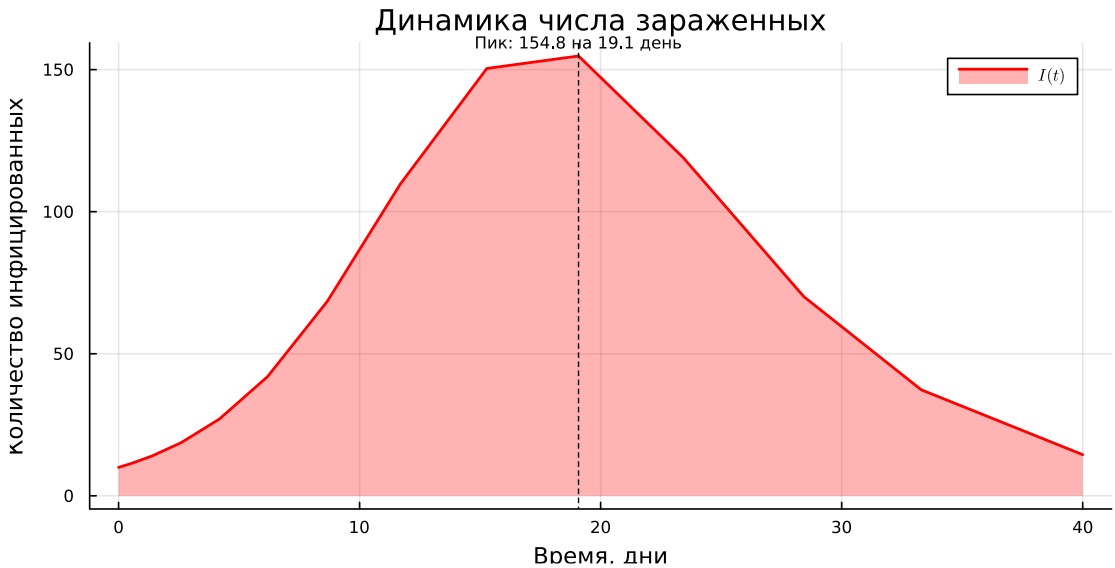


График в логарифмическом масштабе (для анализа экспоненциального роста)

```
                  plt3 = @df df_ode plot(:t, :I,
                                         label=L"I(t)",
                                         xlabel="Время, дни",
                                         ylabel="Количество инфицированных
(лог. масштаб)",
                                         title="Экспоненциальный рост (лог.
шкала)",
                                         yscale=:log10,
                                         color=:red,
                                         linewidth=2,
                                         grid=true,
                                         size=(800, 400))
```

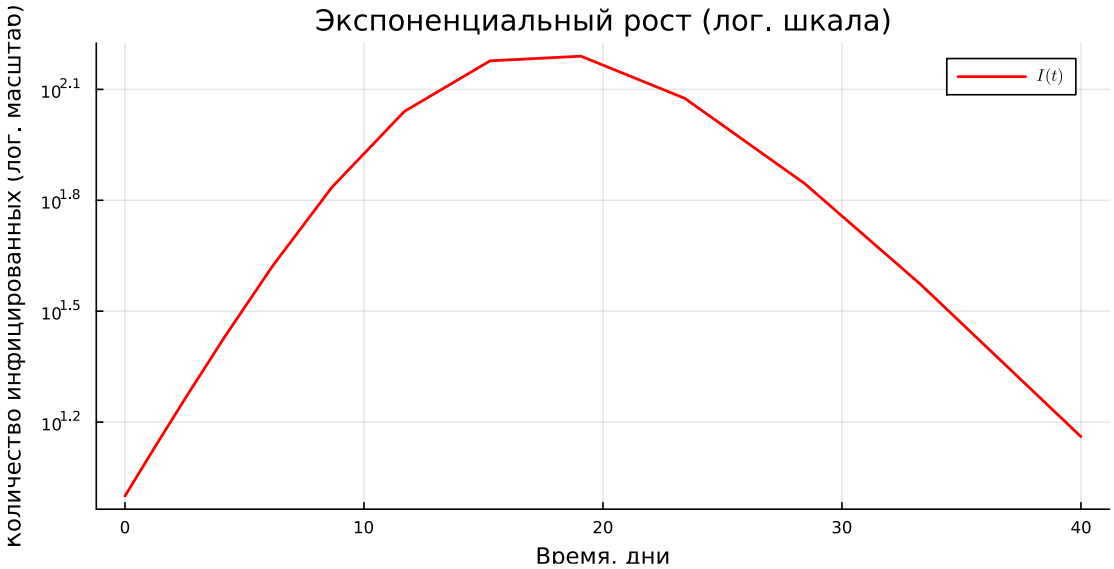
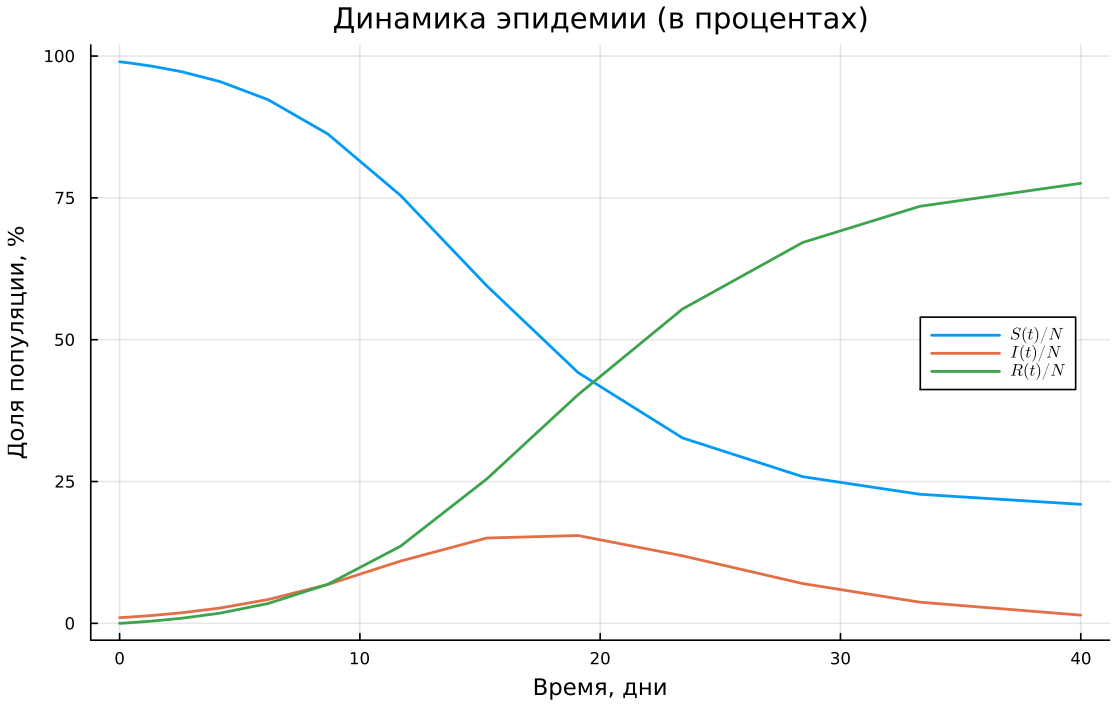


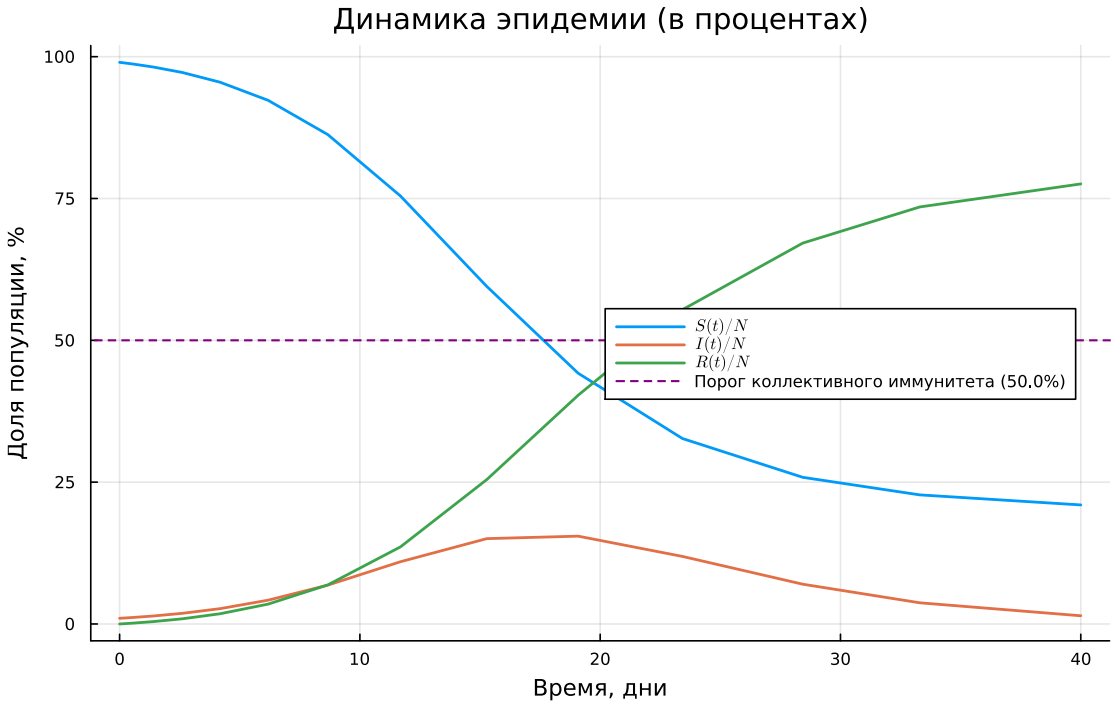
График долей населения (в процентах)

```
plt4 = @df df_ode plot(:t,
                       [:S :I :R] ./ df_ode.N .* 100,
                       label=[L"S(t)/N" L"I(t)/N" L"R(t)/N"],
                       xlabel="Время, дни",
                       ylabel="Доля популяции, %",
                       title="Динамика эпидемии (в процентах)",
                       linewidth=2,
                       legend=:right,
                       grid=true,
                       size=(800, 500))
```



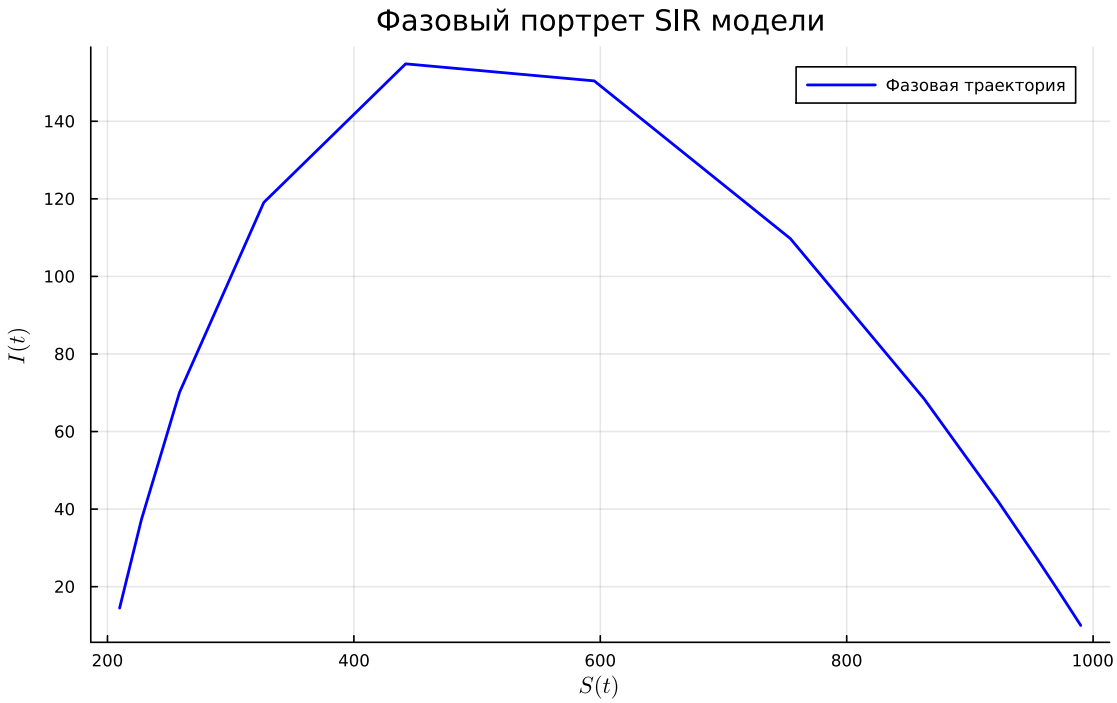
Горизонтальная линия для порога коллективного иммунитета

```
                    if R0 > 1
                        herd_immunity_threshold = (1 - 1/R0) * 100
                        hline!(plt4, [herd_immunity_threshold], color=:purple,
linestyle=:dash,
                               label="Порог коллективного иммунитета
($(round(herd_immunity_threshold, digits=1))%)",
                               linewidth=1.5)
                    end
```



Фазовый портрет (I vs S)

```
                    plt5 = plot(df_ode.S, df_ode.I,
                                label="Фазовая траектория",
                                xlabel=L"S(t)",
                                ylabel=L"I(t)",
                                title="Фазовый портрет SIR модели",
                                color=:blue,
                                linewidth=2,
                                grid=true,
                                size=(800, 500),
                                legend=:topright)
```

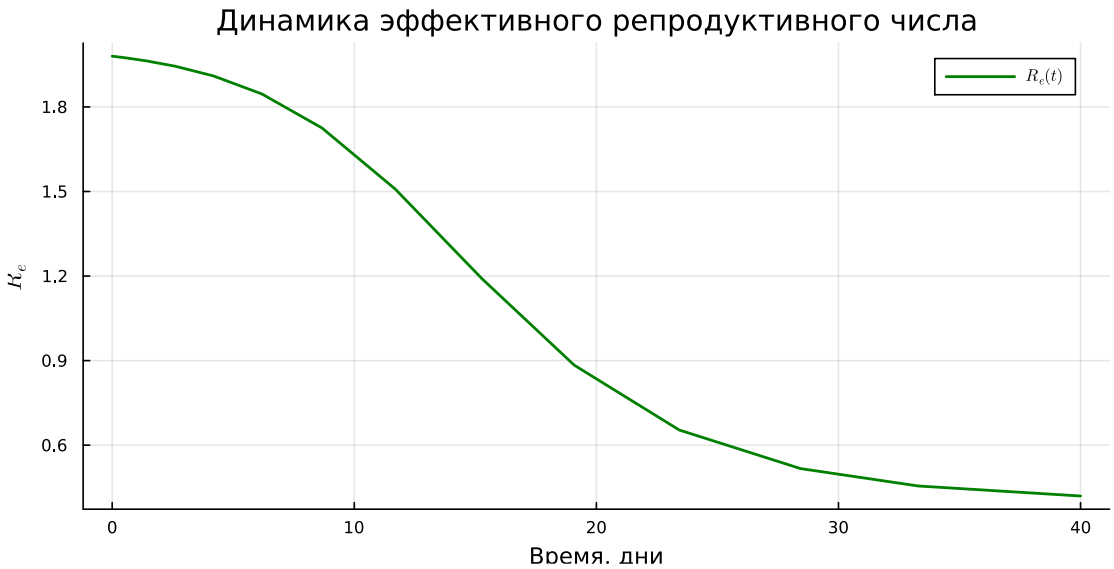


Добавление стрелок направления

```
                    for i in 1:50:length(df_ode.S)-1
                        plot!(plt5, [df_ode.S[i], df_ode.S[i+1]], [df_ode.I[i], df_ode.I[i+1]],
                              arrow=:closed, color=:blue, alpha=0.5, label=false)
                    end
```

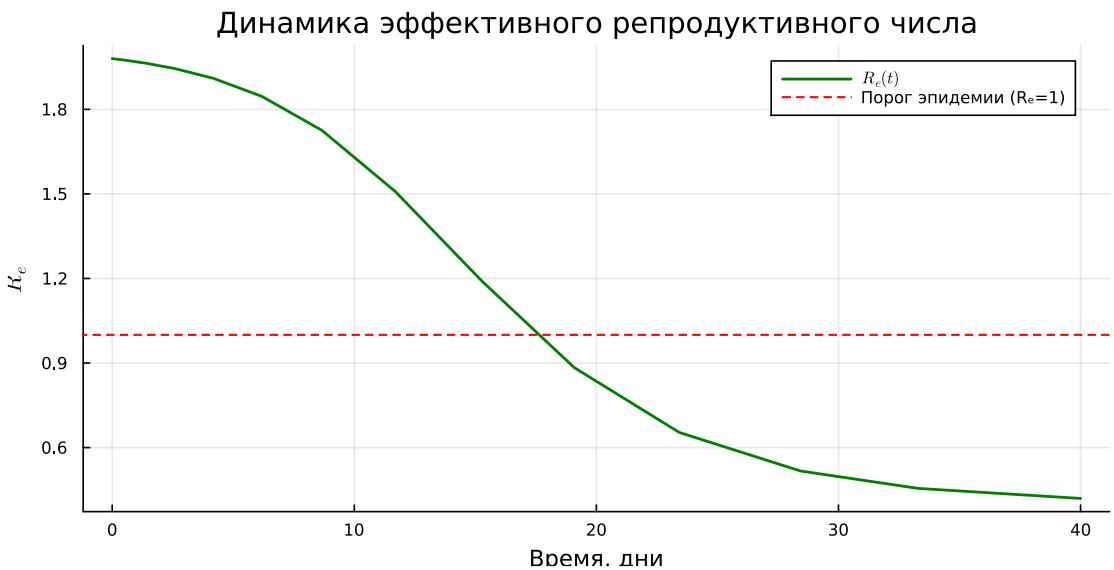
График R_e - эффективного репродуктивного числа

```
                    df_ode[!, :Re] = R0 .* df_ode.S ./ df_ode.N
                    plt6 = @df df_ode plot(:t, :Re,
                                           label=L"R_e(t)",
                                           xlabel="Время, дни",
                                           ylabel=L"R_e",
                                           title="Динамика эффективного репродуктивного числа",
                                           color=:green,
                                           linewidth=2,
                                           grid=true,
                                           size=(800, 400))
```



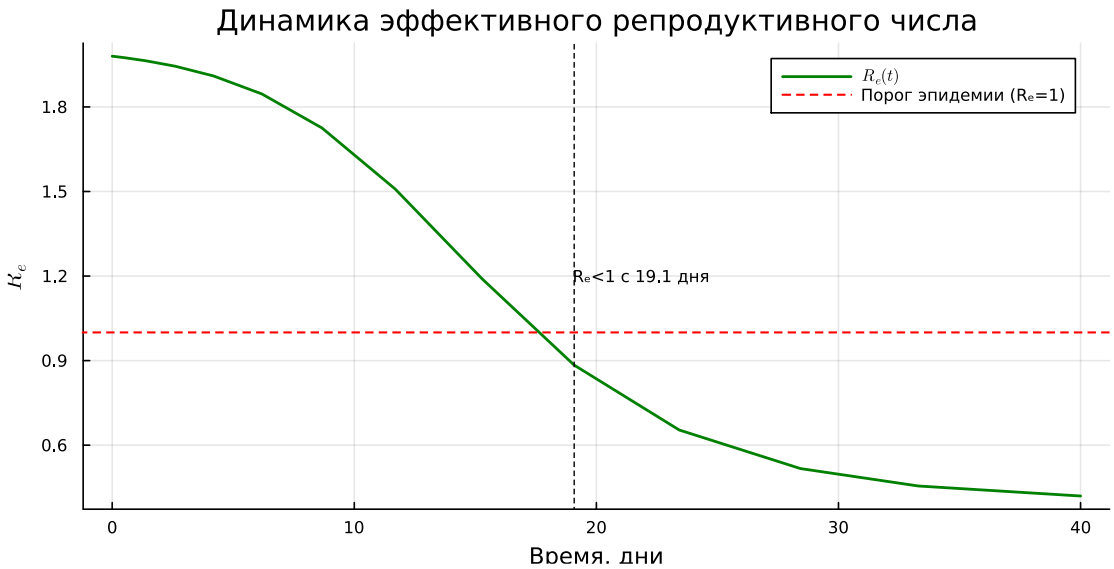
Горизонтальная линия на уровне 1

```
                    hline!(plt6, [1.0], color=:red, linestyle=:dash,
label="Порог эпидемии (Rₑ=1)", linewidth=1.5)
```



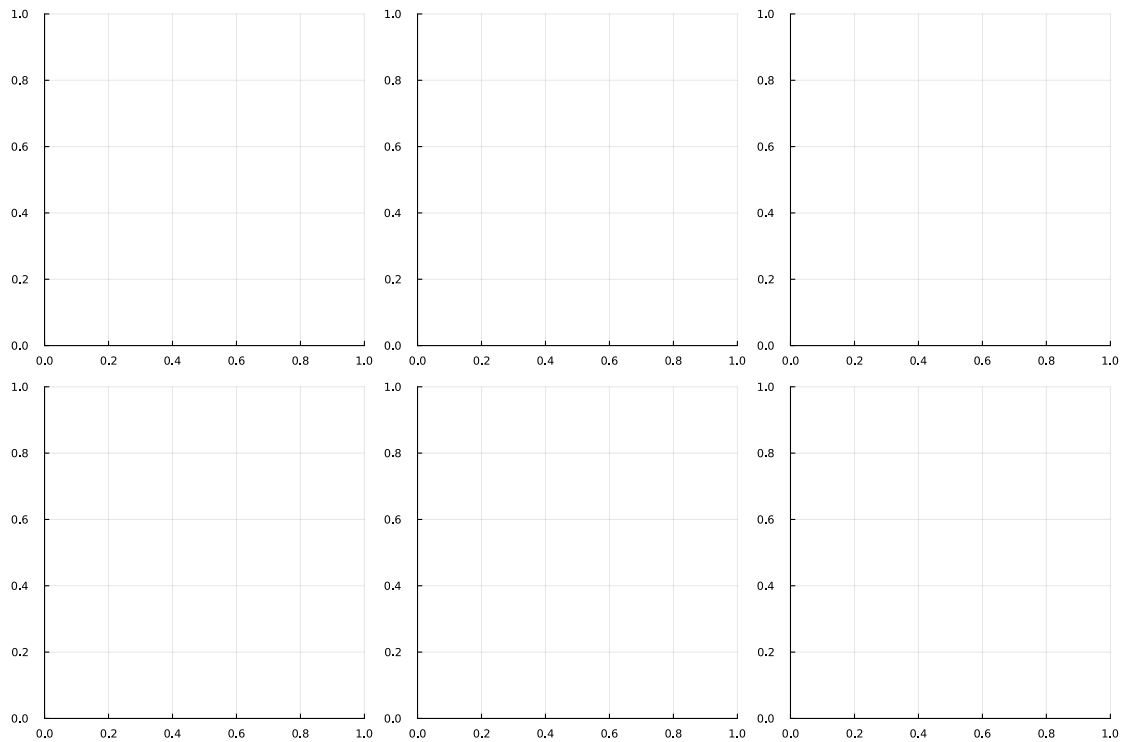
Отметка момента, когда R_e становится < 1

```
                    cross_idx = findfirst(x -> x < 1, df_ode.Re)
                    if !isnothing(cross_idx) && cross_idx > 1
                        cross_time = df_ode.t[cross_idx]
                        vline!(plt6, [cross_time], color=:black,
linestyle=:dash, label=false, linewidth=1)
                        annotate!(plt6, cross_time, 1.2,
                                  text("Rₑ<1 с $(round(cross_time, digits=1))
дня", 8, :left))
                    end
```



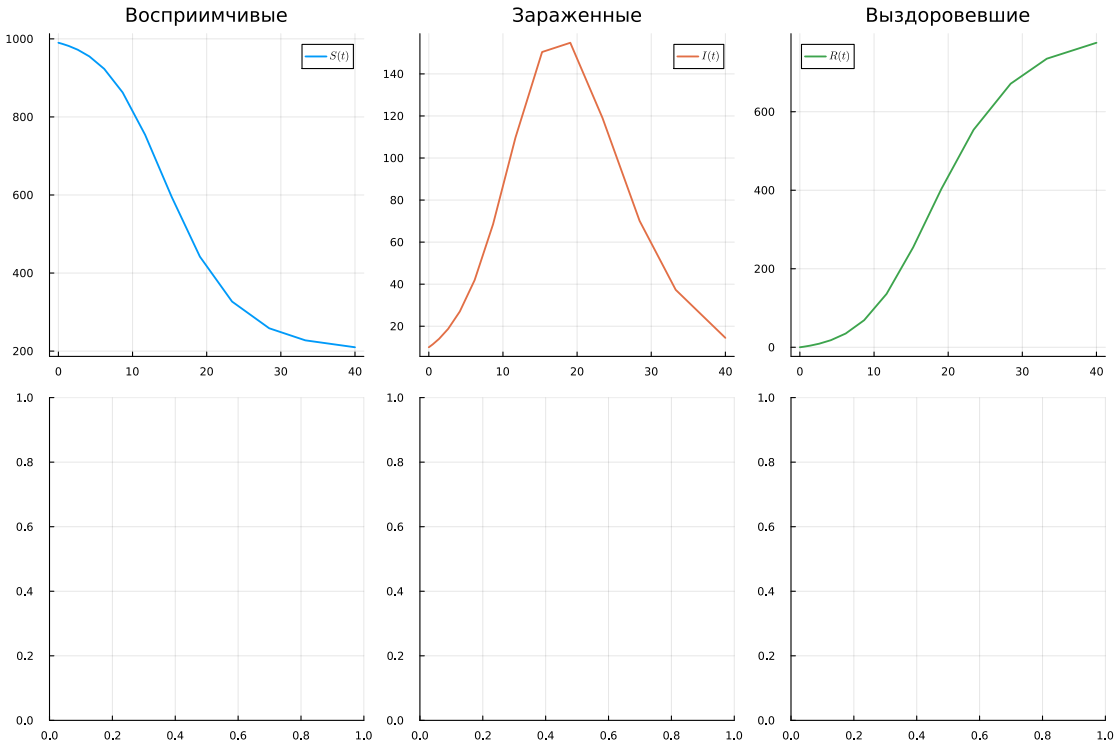
Компактный график всех кривых в одной панели

```
plt7 = plot(layout=(2, 3), size=(1200, 800))
```



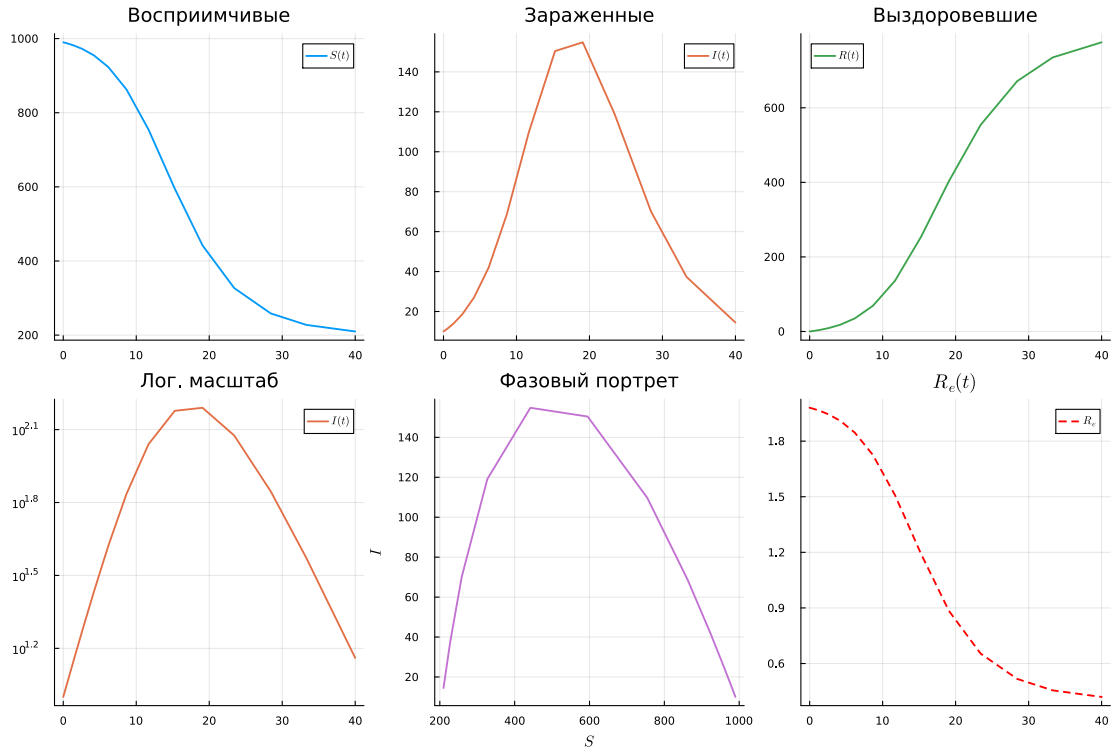
Верхний ряд

```
plot!(plt7[1], df_ode.t, df_ode.S, label=L"S(t)", color=1, linewidth=2, title="Восприимчивые")
plot!(plt7[2], df_ode.t, df_ode.I, label=L"I(t)", color=2, linewidth=2, title="Зараженные")
plot!(plt7[3], df_ode.t, df_ode.R, label=L"R(t)", color=3, linewidth=2, title="Выздоровевшие")
```



Нижний ряд

```
                    plot!(plt7[4], df_ode.t, df_ode.I, label=L"I(t)", color=2,
linewidth=2,
                          yscale=:log10, title="Лог. масштаб")
                    plot!(plt7[5], df_ode.S, df_ode.I, label=false, color=4,
linewidth=2,
                          title="Фазовый портрет", xlabel=L"S", ylabel=L"I")
                    plot!(plt7[6], df_ode.t, df_ode.Re, label=L"R_e",
color=:green, linewidth=2,
                          title=L"R_e(t)", hline=[1.0], linestyle=:dash,
linecolor=:red)
```



Сохранение графиков

```
savefig(plt1, plotsdir(script_name, "sir_main.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
savefig(plt5, plotsdir(script_name,
"sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))
savefig(plt7, plotsdir(script_name, "sir_panel.png"))
```

```
"/home/kali/work/study/2026-1/2026-1==simulation-modeling/2026-1--study--
simulation-modeling/labs/lab02/project/plots/sir_panel.png"
```

Бенчмарк для оценки производительности

```
println("\nБенчмарк решения:")
@benchmark solve(prob_ode, dt = δt)
```

```
Бенчмарк решения:

BenchmarkTools.Trial: 10000 samples with 5 evaluations per sample.
 Range (min … max):   5.630 μs …  16.415 ms  ┊ GC (min … max): 0.00% … 0.00%
 Time  (median):      9.029 μs               ┊ GC (median):    0.00%
 Time  (mean ± σ):   13.446 μs ± 180.645 μs  ┊ GC (mean ± σ):  0.00% ± 0.00%
```

```
[redacted]
  5.63 μs       Histogram: log(frequency) by time      44.3 μs <

 Memory estimate: 17.39 KiB, allocs estimate: 333.
```

Дополнительный анализ

```
                    println("\n=== АНАЛИЗ РЕЗУЛЬТАТОВ ===")
                    println("Общая численность популяции (контроль): N = ",
round(df_ode.N[1], digits=1))
                    println("Пиковое число зараженных: I_max = ",
round(peak_value, digits=1))
                    println("Время достижения пика: t_peak = ", round(peak_time,
digits=1), " дней")
                    println("Итоговое число переболевших: R(∞) = ",
round(df_ode.R[end], digits=1))
                    println("Доля переболевших: ",
round(df_ode.R[end]/df_ode.N[1]*100, digits=1), "%")

                    if R0 > 1
                        println("\nТеоретический анализ:")
                        println("  - Порог коллективного иммунитета: ",
round((1-1/R0)*100, digits=1), "%")
                        println("  - Теоретический пик при S/N = 1/R₀ = ",
round(1/R0, digits=3))
                    end


=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Общая численность популяции (контроль): N = 1000.0
Пиковое число зараженных: I_max = 154.8
Время достижения пика: t_peak = 19.1 дней
Итоговое число переболевших: R(∞) = 775.7
Доля переболевших: 77.6%

Теоретический анализ:
  - Порог коллективного иммунитета: 50.0%
  - Теоретический пик при S/N = 1/R₀ = 0.5
```

6. Модель Лотки–Вольтерры

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate "project"
using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
default(fmt = :png)
using Statistics
```

```julia
using FFTW

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
  Activating project at `~/work/study/2026-1/2026-1==simulation-modeling/2026-1-
-study--simulation-modeling/labs/lab02/project`
┌ Warning: DrWatson could not find find a project file by recursively checking
given `dir` and its parents. Returning `nothing` instead.
│ (given dir: /home/kali)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

```
"/home/kali/work/study/2026-1/2026-1==simulation-modeling/2026-1--study--
simulation-modeling/labs/lab02/project/data/"
```

Описание модели Лотки-Вольтерры

```julia
"""
Модель Лотки-Вольтерры (хищник-жертва)

Система уравнений:
dx/dt = αx - βxy  # Изменение популяции жертв
dy/dt = δxy - γy  # Изменение популяции хищников

Где:
x - популяция жертв (например, зайцы)
y - популяция хищников (например, лисы)
α - естественный прирост жертв (в отсутствие хищников)
β - коэффициент поедания жертв хищниками
δ - коэффициент прироста хищников за счет поедания жертв
γ - естественная смертность хищников (в отсутствие жертв)
"""

function lotka_volterra!(du, u, p, t)
    x, y = u          # x - жертвы, y - хищники
    α, β, δ, γ = p    # параметры модели

    @inbounds begin
        du[1] = α*x - β*x*y      # уравнение для жертв
        du[2] = δ*x*y - γ*y      # уравнение для хищников
    end
    nothing
end
```

```
lotka_volterra! (generic function with 1 method)
```

Параметры модели и начальные условия Классические параметры из литературы

```julia
p_lv = [0.1,   # α: скорость размножения жертв
        0.02,  # β: скорость поедания жертв хищниками
        0.01,  # δ: коэффициент конверсии пищи (жертв) в хищников
        0.3]   # γ: смертность хищников
```

```
4-element Vector{Float64}:
 0.1
 0.02
 0.01
 0.3
```

Начальные условия: [жертвы, хищники]

```julia
u0_lv = [40.0, 9.0]  # начальная популяция
```

```
2-element Vector{Float64}:
 40.0
  9.0
```

Временные параметры

```julia
tspan_lv = (0.0, 200.0)  # длительность симуляции
dt_lv = 0.01              # шаг интегрирования
```

```
0.01
```

Создание и решение задачи

```julia
prob_lv = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_lv)
sol_lv = solve(prob_lv,
               dt = dt_lv,
               Tsit5(),           # Метод 5-го порядка
               reltol=1e-8,       # Относительная точность
               abstol=1e-10,      # Абсолютная точность
               saveat=0.1,        # Сохраняем каждые 0.1 единицы времени
               dense=true         # Включаем плотный вывод для интерполяции
               )
```

```
┌ Warning: Verbosity toggle: dense_output_saveat
│  Dense output is incompatible with saveat. Please use the SavingCallback from
the Callback Library to mix the two behaviors.
└ @ OrdinaryDiffEqCore
~/.julia/packages/OrdinaryDiffEqCore/tCI9o/src/solve.jl:133

retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 2001-element Vector{Float64}:
   0.0
   0.1
   0.2
   0.3
   0.4
   0.5
   0.6
   0.7
   0.8
   0.9
   1.0
   1.1
```

```
   1.2
   ⋮
 198.9
 199.0
 199.1
 199.2
 199.3
 199.4
 199.5
 199.6
 199.7
 199.8
 199.9
 200.0
u: 2001-element Vector{Vector{Float64}}:
 [40.0, 9.0]
 [39.67773201890398, 9.088990237207573]
 [39.3511378950319, 9.1758827263069]
 [39.02053587808413, 9.260562174341601]
 [38.6862482028163, 9.342916311941519]
 [38.348600187590776, 9.422836265416295]
 [38.007919320048416, 9.500216918705295]
 [37.664534369324684, 9.574957250331227]
 [37.318774466549755, 9.646960666854303]
 [36.970968217675335, 9.71613530831863]
 [36.62144287330037, 9.782394321389809]
 [36.27052336512943, 9.845656154265615]
 [35.91853163310632, 9.905844751307407]
 ⋮
 [17.88967563421508, 5.712401894065408]
 [17.865436242316996, 5.643570483728569]
 [17.84367345387529, 5.575440219897811]
 [17.824353144929567, 5.50801931850314]
 [17.807442037033947, 5.441315154549741]
 [17.79290768632291, 5.375334300035647]
 [17.780718473177817, 5.310082577781689]
 [17.770843588537023, 5.245565084818681]
 [17.763253013148027, 5.181786245480113]
 [17.757917517460715, 5.118749811715716]
 [17.75480865796634, 5.056458873592463]
 [17.75389874795947, 4.994915937053484]
```

Подготовка данных

```
df_lv = DataFrame()
df_lv[!, :t] = sol_lv.t
df_lv[!, :prey] = [u[1] for u in sol_lv.u]  # жертвы
    df_lv[!, :predator] = [u[2] for u in sol_lv.u]  # хищники
```

```
2001-element Vector{Float64}:
 9.0
 9.088990237207573
 9.1758827263069
```

```
 9.260562174341601
 9.342916311941519
 9.422836265416295
 9.500216918705295
 9.574957250331227
 9.646960666854303
 9.71613530831863
 9.782394321389809
 9.845656154265615
 9.905844751307407
 ⋮
 5.712401894065408
 5.643570483728569
 5.575440219897811
 5.50801931850314
 5.441315154549741
 5.375334300035647
 5.310082577781689
 5.245565084818681
 5.181786245480113
 5.118749811715716
 5.056458873592463
 4.994915937053484
```

Рассчет производных для анализа

```
        df_lv[!, :dprey_dt] = p_lv[1] .* df_lv.prey .- p_lv[2] .* df_lv.prey .*
df_lv.predator
        df_lv[!, :dpredator_dt] = p_lv[3] .* df_lv.prey .* df_lv.predator .-
p_lv[4] .* df_lv.predator
```

```
2001-element Vector{Float64}:
  0.9000000000000004
  0.8796081183812947
  0.8580494468233706
  0.8353523334487725
  0.8115489002366512
  0.7866749261309169
  0.7607697060795098
  0.733875889299783
  0.7060392940838303
  0.6773087043292239
  0.6477356516318054
  0.6173741696035293
  0.5862805551325314
  ⋮
 -0.6917903984485623
 -0.6848226585578221
 -0.6777687195147514
 -0.6706369809299996
 -0.6634355041661331
 -0.6561720201740996
 -0.6488539394858811
```

```
 -0.6414883588875672
 -0.6340820722588977
 -0.6266415740250623
 -0.6191730642026372
 -0.6116824631058786
```

Вывод информации о модели

```
        println("="^60)
        println("Модель Лотки-Вольтерры (хищник-жертва)")
        println("="^60)
        println("\nПараметры модели:")
        println("α (скорость размножения жертв) = ", p_lv[1])
        println("β (скорость поедания жертв) = ", p_lv[2])
        println("δ (коэффициент конверсии) = ", p_lv[3])
        println("γ (смертность хищников) = ", p_lv[4])
        println("\nНачальные условия:")
        println("Жертвы (x₀) = ", u0_lv[1])
        println("Хищники (y₀) = ", u0_lv[2])
```

```
============================================================
Модель Лотки-Вольтерры (хищник-жертва)
============================================================


Параметры модели:
α (скорость размножения жертв) = 0.1
β (скорость поедания жертв) = 0.02
δ (коэффициент конверсии) = 0.01
γ (смертность хищников) = 0.3

Начальные условия:
Жертвы (x₀) = 40.0
Хищники (y₀) = 9.0
```

Стационарные точки (нулевые изоклины)

```
        x_star = p_lv[4] / p_lv[3]  # стационарная точка для жертв
        y_star = p_lv[1] / p_lv[2]  # стационарная точка для хищников
        println("\nСтационарные точки (положения равновесия):")
        println("x* = γ/δ = ", round(x_star, digits=3))
        println("y* = α/β = ", round(y_star, digits=3))
```

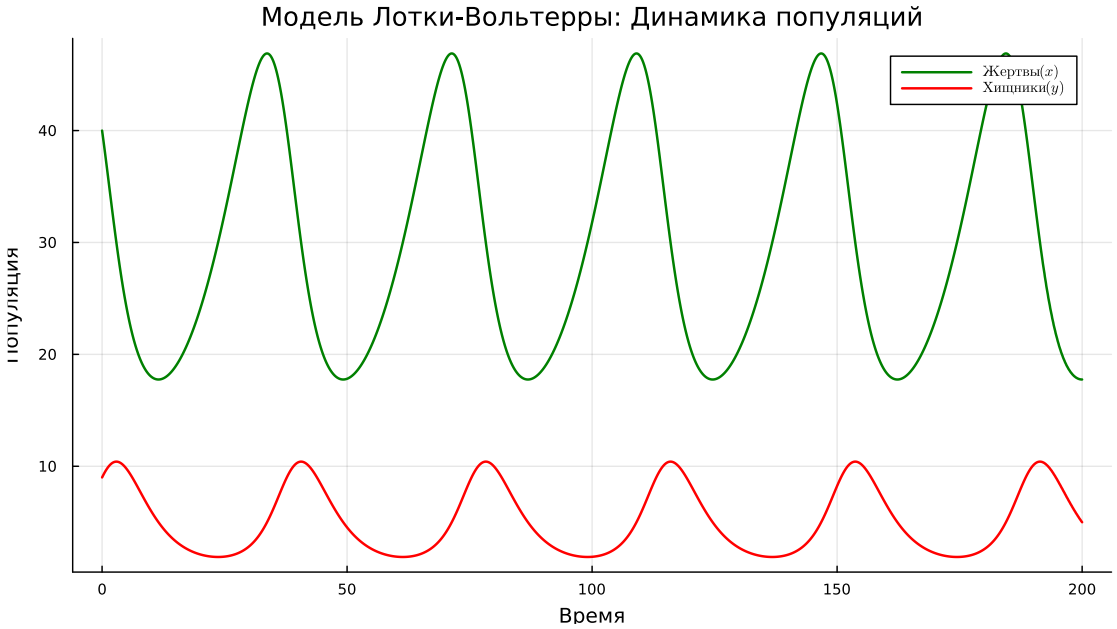
```


Стационарные точки (положения равновесия):
x* = γ/δ = 30.0
y* = α/β = 5.0
```

Построение графиков

График 1: Динамика популяций во времени

```
        plt1 = plot(df_lv.t, [df_lv.prey df_lv.predator],
                    label=[L"Жертвы (x)" L"Хищники (y)"],
                    xlabel="Время",
```

```
                    ylabel="Популяция",
                    title="Модель Лотки-Вольтерры: Динамика популяций",
                    linewidth=2,
                    legend=:topright,
                    grid=true,
                    size=(900, 500),
                    color=[:green :red])
```



Добавление стационарных уровней

```
        hline!(plt1, [x_star], color=:green, linestyle=:dash, alpha=0.5,
label="x* (равновесие жертв)")
        hline!(plt1, [y_star], color=:red, linestyle=:dash, alpha=0.5, label="y*
(равновесие хищников)")
```

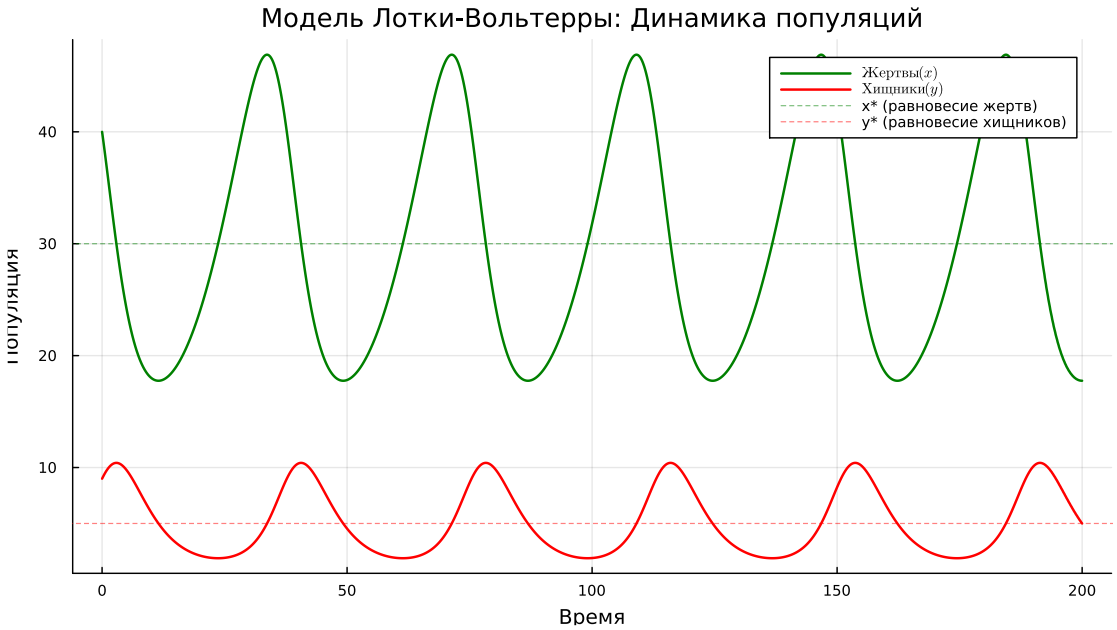
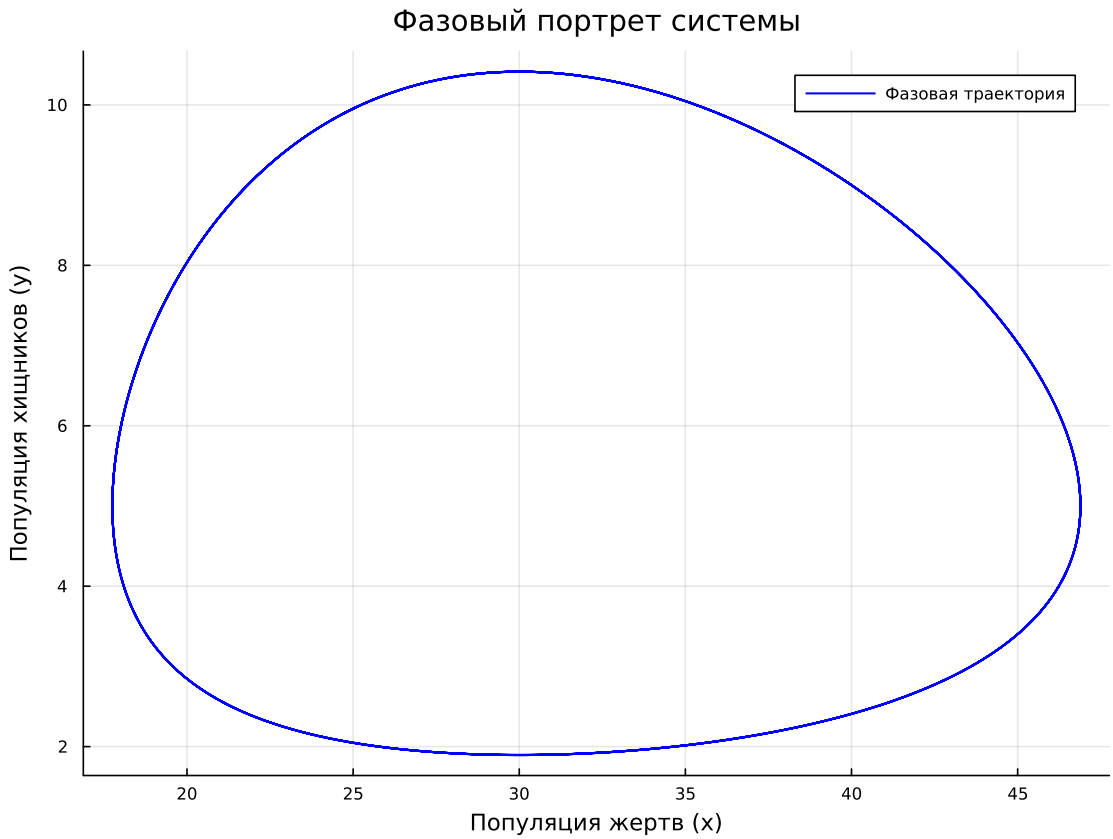


График 2: Фазовый портрет (хищники vs жертвы)

```
plt2 = plot(df_lv.prey, df_lv.predator,
            label="Фазовая траектория",
            xlabel="Популяция жертв (x)",
            ylabel="Популяция хищников (y)",
            title="Фазовый портрет системы",
            color=:blue,
            linewidth=1.5,
            grid=true,
            size=(800, 600),
            legend=:topright)
```

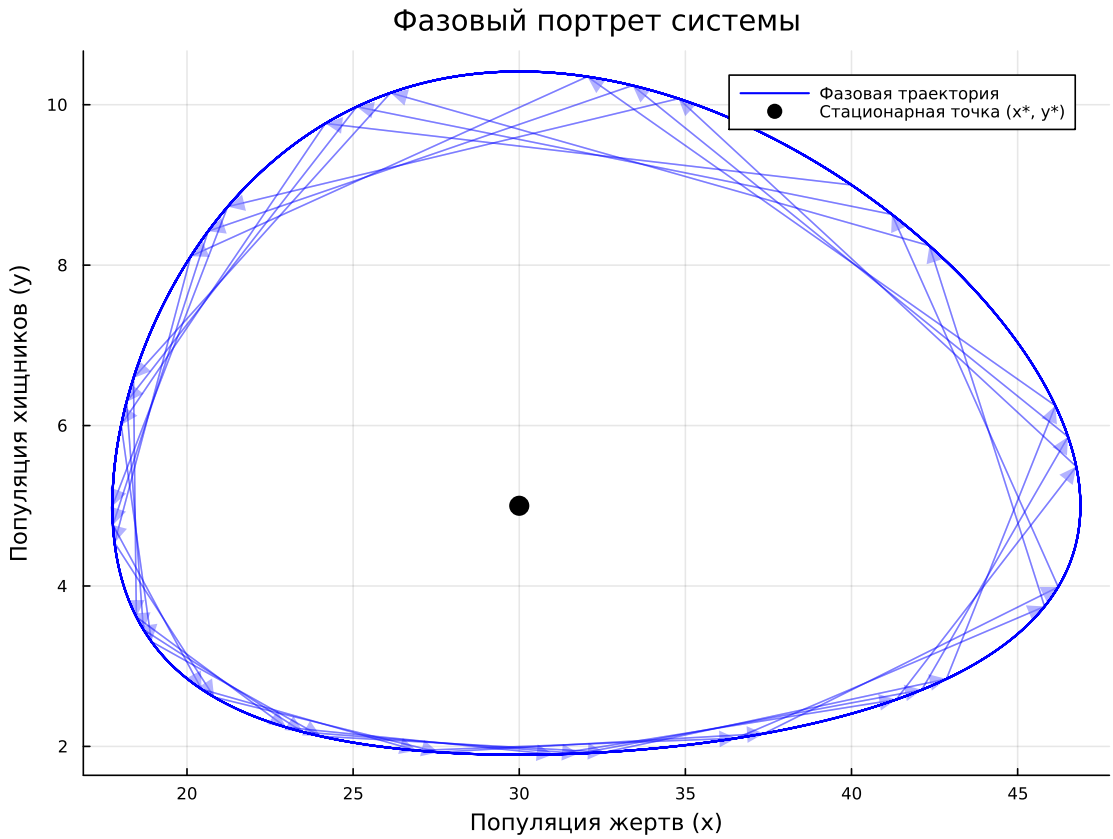


Добавление стрелок направления на фазовом портрете

```
step = 50  # шаг для отображения стрелок
for i in 1:step:length(df_lv.prey)-step
    plot!(plt2, [df_lv.prey[i], df_lv.prey[i+step]],
          [df_lv.predator[i], df_lv.predator[i+step]],
          arrow=:closed, color=:blue, alpha=0.3, label=false)
end
```

Добавление стационарной точки

```
scatter!(plt2, [x_star], [y_star],
         color=:black, markersize=8, label="Стационарная точка (x*,
y*)")
```



Изоклины (нулевого роста)

```
        x_range = LinRange(0, maximum(df_lv.prey)*1.1, 100)
        y_nullcline = p_lv[1] ./ (p_lv[2] .* x_range)  # y-изоклина (dy/dt = 0)
        plot!(plt2, x_range, y_nullcline,
              color=:red, linestyle=:dash, linewidth=1.5, label="Изоклина
хищников (dy/dt=0)")

        y_range = LinRange(0, maximum(df_lv.predator)*1.1, 100)
        x_nullcline = p_lv[4] ./ (p_lv[3] .* ones(length(y_range)))  # x-
изоклина (dx/dt = 0)
        plot!(plt2, x_nullcline, y_range,
              color=:green, linestyle=:dash, linewidth=1.5, label="Изоклина
жертв (dx/dt=0)")
```

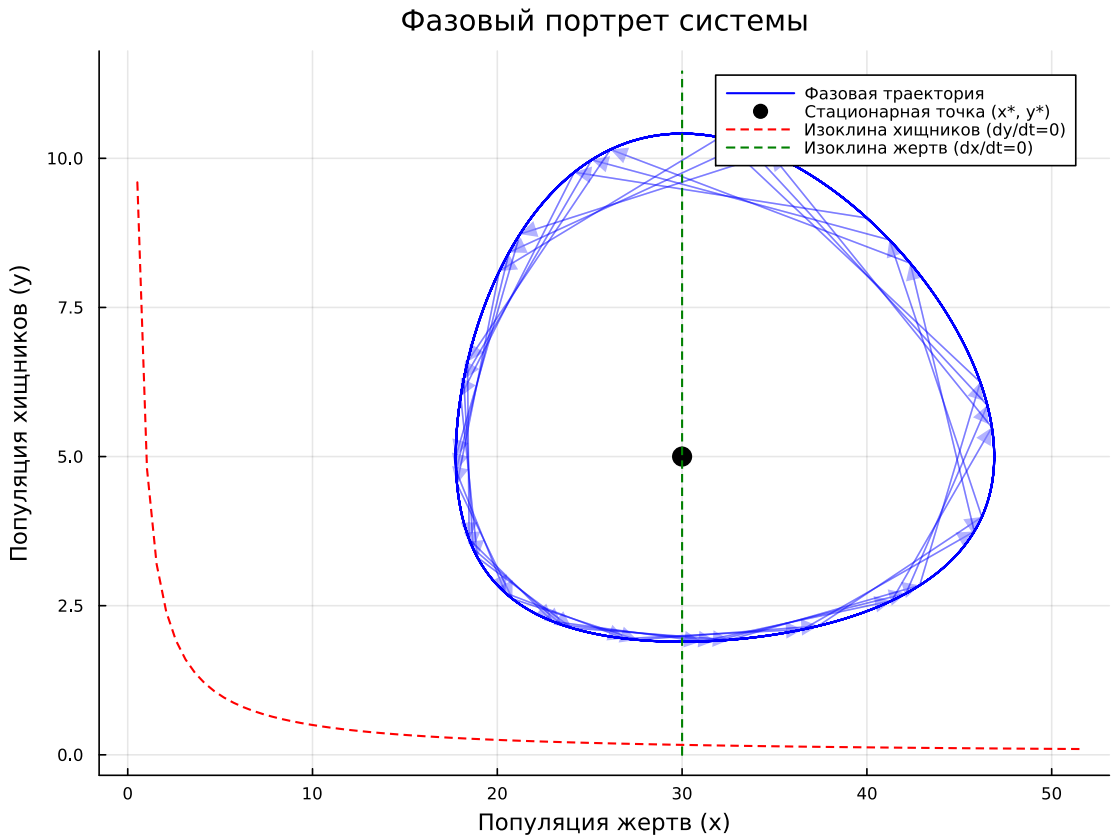


График 3: Производные (скорости изменения)

```
        plt3 = plot(df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
                    label=[L"dx/dt" L"dy/dt"],
                    xlabel="Время",
                    ylabel="Скорость изменения",
                    title="Производные популяций",
                    linewidth=1.5,
                    legend=:topright,
                    grid=true,
                    size=(900, 400),
                    color=[:green :red])

        hline!(plt3, [0], color=:black, linestyle=:solid, alpha=0.3,
label=false)
```

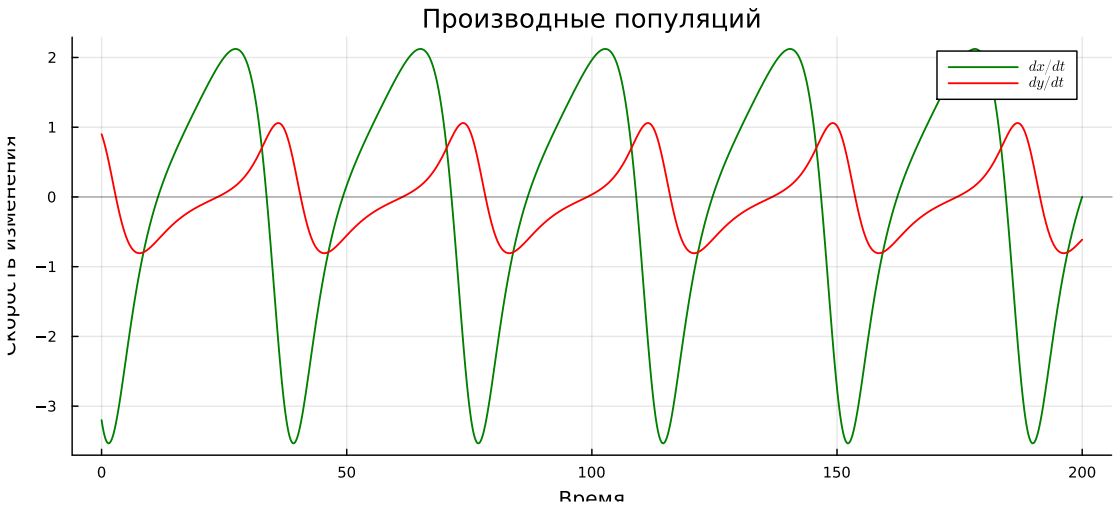


График 4: Относительные изменения (в %)

```
        df_lv[!, :prey_pct_change] = df_lv.dprey_dt ./ df_lv.prey .* 100
        df_lv[!, :predator_pct_change] = df_lv.dpredator_dt ./ df_lv.predator .*
100

        plt4 = plot(df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_change],
                    label=[L"dx/dt / x (\%)" L"dy/dt / y (\%)"],
                    xlabel="Время",
                    ylabel="Относительное изменение, %",
                    title="Относительные темпы роста",
                    linewidth=1.5,
                    legend=:topright,
                    grid=true,
                    size=(900, 400),
                    color=[:green :red])
```

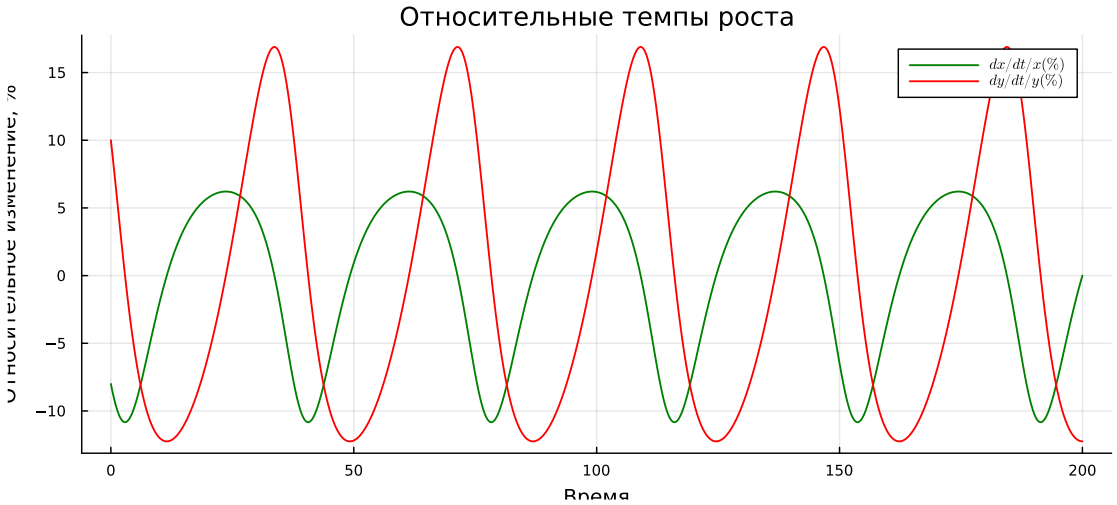


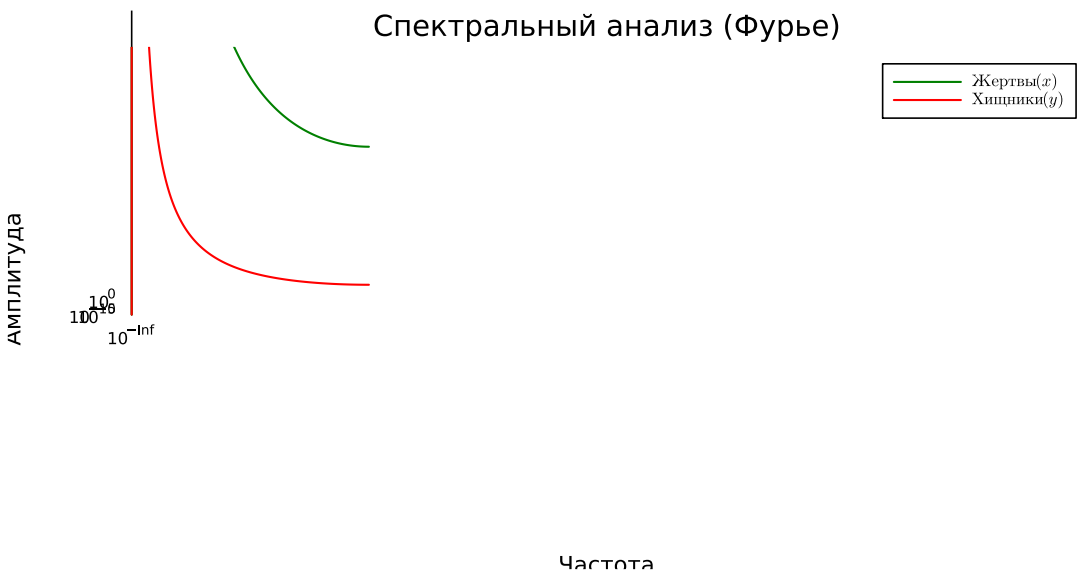
График 5: Спектральный анализ (быстрое преобразование Фурье)

```
        function compute_fft(signal, dt)
            n = length(signal) # Используем rfft для вещественных сигналов
(возвращает только положительные частоты)
```

```
            spectrum = abs.(rfft(signal)) # Соответствующие частоты для rfft
            freq = rfftfreq(n, 1/dt)
            return freq, spectrum
        end

compute_fft (generic function with 1 method)
```

Вычисление периодов колебаний

```
        freq_prey, spectrum_prey = compute_fft(df_lv.prey .- mean(df_lv.prey),
dt_lv)
        freq_predator, spectrum_predator = compute_fft(df_lv.predator .-
mean(df_lv.predator), dt_lv)

        plt5 = plot(freq_prey, [spectrum_prey spectrum_predator],
                    label=[L"Жертвы (x)" L"Хищники (y)"],
                    xlabel="Частота",
                    ylabel="Амплитуда",
                    title="Спектральный анализ (Фурье)",
                    linewidth=1.5,
                    xscale=:log10,
                    yscale=:log10,
                    legend=:topright,
                    grid=true,
                    size=(800, 400),
                    color=[:green :red])
```




```
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
```

```
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
```

Нахождение доминирующих частот

```julia
        if length(spectrum_prey) > 0
            idx_prey = argmax(spectrum_prey[2:end]) + 1  # пропускаем нулевую
частоту
            dominant_freq_prey = freq_prey[idx_prey]
            period_prey = 1/dominant_freq_prey
            println("\nДоминирующая частота колебаний жертв: ",
round(dominant_freq_prey, digits=4), " Гц")
            println("Период колебаний жертв: ", round(period_prey, digits=2), "
единиц времени")
        end
```

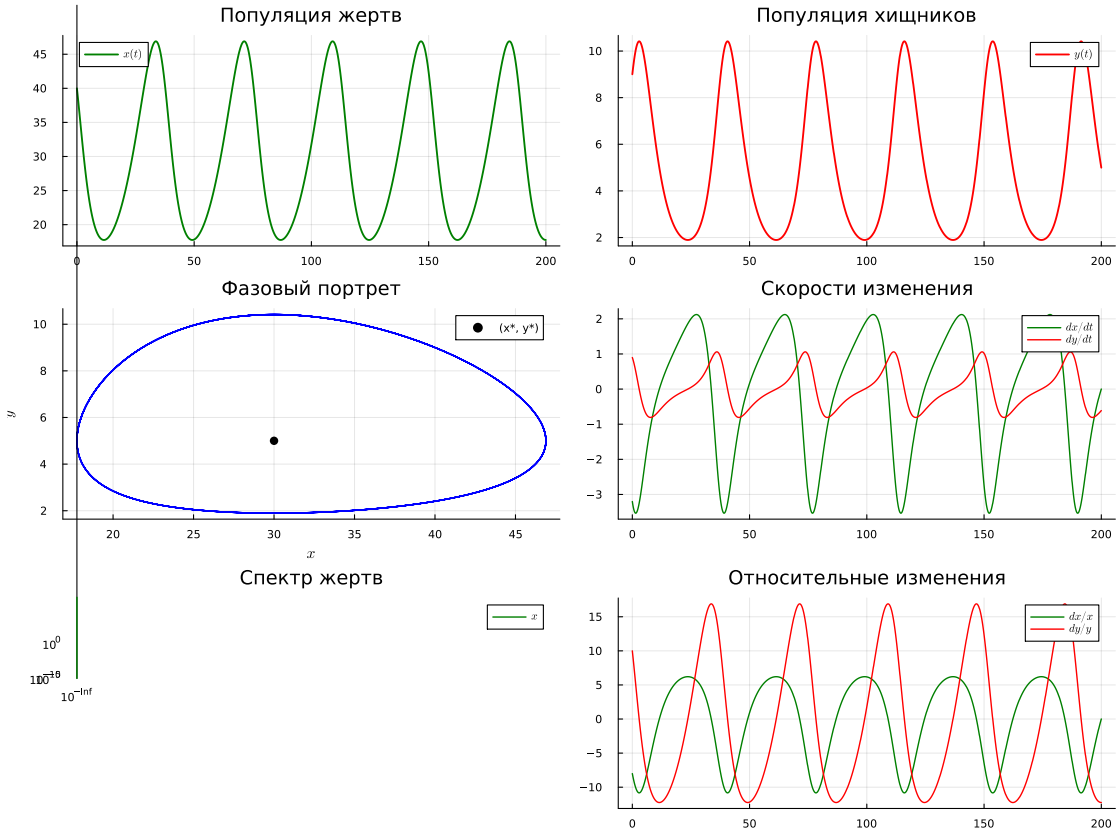
```
Доминирующая частота колебаний жертв: 0.2499 Гц
Период колебаний жертв: 4.0 единиц времени
```

График 6: Компактная панель всех графиков

```julia
        plt6 = plot(layout=(3, 2), size=(1200, 900))

        plot!(plt6[1], df_lv.t, df_lv.prey, label=L"x(t)", color=:green,
linewidth=2,
              title="Популяция жертв", grid=true)
        plot!(plt6[2], df_lv.t, df_lv.predator, label=L"y(t)", color=:red,
linewidth=2,
              title="Популяция хищников", grid=true)
        plot!(plt6[3], df_lv.prey, df_lv.predator, label=false, color=:blue,
linewidth=1.5,
```

```
              title="Фазовый портрет", xlabel=L"x", ylabel=L"y", grid=true)
        scatter!(plt6[3], [x_star], [y_star], color=:black, markersize=5,
label="(x*, y*)")
        plot!(plt6[4], df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
              label=[L"dx/dt" L"dy/dt"], color=[:green :red], linewidth=1.5,
              title="Скорости изменения", grid=true, legend=:topright)
        plot!(plt6[5], freq_prey, spectrum_prey, label=L"x", color=:green,
linewidth=1.5,
              title="Спектр жертв", xscale=:log10, yscale=:log10, grid=true)
        plot!(plt6[6], df_lv.t, [df_lv.prey_pct_change
df_lv.predator_pct_change],
              label=[L"dx/x" L"dy/y"], color=[:green :red], linewidth=1.5,
              title="Относительные изменения", grid=true, legend=:topright)
```

```
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
```




```
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
```

Анализ результатов

```julia
        println("\n" * "="^60)
        println("Анализ результатов")
        println("="^60)

        println("\nОсновные статистики:")
        println("Жертвы: min = ", round(minimum(df_lv.prey), digits=2),
                ", max = ", round(maximum(df_lv.prey), digits=2),
                ", mean = ", round(mean(df_lv.prey), digits=2))
        println("Хищники: min = ", round(minimum(df_lv.predator), digits=2),
                ", max = ", round(maximum(df_lv.predator), digits=2),
                ", mean = ", round(mean(df_lv.predator), digits=2))
```

```
============================================================
Анализ результатов
============================================================

Основные статистики:
```

```
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2
```

Упрощенный анализ колебаний без поиска максимумов Вместо сложного анализа сдвига фаз, просто посчитаем основные характеристики

Находим время первого пика жертв (простой алгоритм)

```
        function find_first_peak(signal, time)
            for i in 2:length(signal)-1
                if signal[i] > signal[i-1] && signal[i] > signal[i+1]
                    return time[i], signal[i]
                end
            end
            return NaN, NaN
        end

        peak_time_prey, peak_value_prey = find_first_peak(df_lv.prey, df_lv.t)
        peak_time_predator, peak_value_predator =
find_first_peak(df_lv.predator, df_lv.t)

        if !isnan(peak_time_prey) && !isnan(peak_time_predator)
            phase_shift = peak_time_predator - peak_time_prey
            println("\nАнализ колебаний:")
            println("Первый пик жертв: время = ", round(peak_time_prey,
digits=2),
                    ", значение = ", round(peak_value_prey, digits=2))
            println("Первый пик хищников: время = ", round(peak_time_predator,
digits=2),
                    ", значение = ", round(peak_value_predator, digits=2))
            println("Сдвиг фаз (хищники отстают): ", round(phase_shift,
digits=2))
        end


Анализ колебаний:
Первый пик жертв: время = 33.7, значение = 46.89
Первый пик хищников: время = 2.9, значение = 10.41
Сдвиг фаз (хищники отстают): -30.8
```

Сохранение графиков

```
        savefig(plt1, plotsdir(script_name, "lv_dynamics.png"))
        savefig(plt2, plotsdir(script_name, "lv_phase_portrait.png"))
        savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
        savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
        savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
        savefig(plt6, plotsdir(script_name, "lv_panel.png"))

┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
```

```
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10
based xscale
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105

"/home/kali/work/study/2026-1/2026-1==simulation-modeling/2026-1--study--
simulation-modeling/labs/lab02/project/plots/lv_panel.png"
```

Дополнительный анализ: чувствительность к параметрам

```julia
        println("\n\n" * "="^60)
        println("Анализ чувствительности")
        println("="^60)
```

```
============================================================
Анализ чувствительности
============================================================
```

Функция для анализа влияния параметров

```julia
        function analyze_parameter_sensitivity(param_index, values, param_name)
            println("\nАнализ чувствительности к параметру: ", param_name)
            results = []

            for val in values
                p_test = copy(p_lv)
                p_test[param_index] = val
                prob_test = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_test)
                sol_test = solve(prob_test, dt = dt_lv)
                prey_end = sol_test.u[end][1]
                predator_end = sol_test.u[end][2]

                push!(results, (param=val, prey=prey_end,
predator=predator_end))
                println("  $(param_name)=$(val): жертвы=$(round(prey_end,2)),
хищники=$(round(predator_end,2))")
            end
            return results
        end
```

```
analyze_parameter_sensitivity (generic function with 1 method)
```

Анализ чувствительности к ключевым параметрам

```
        if false  # Установите в true для выполнения анализа
            println("\n1. Влияние скорости размножения жертв (α):")
            analyze_parameter_sensitivity(1, [0.05, 0.1, 0.2, 0.3], "α")

            println("\n2. Влияние смертности хищников (γ):")
            analyze_parameter_sensitivity(4, [0.1, 0.3, 0.5, 0.7], "γ")
        end

        println("\nМоделирование завершено успешно!")


Моделирование завершено успешно!
```

7. Выводы

В ходе данной лабораторной работы мной были изучены основные модели SIR и Лотки–Вольтерры, а так же аспекты их программной реализации.

8. Список литературы

1. Kermack W.O., McKendrick A.G. A Contribution to the Mathematical Theory of Epidemics // Proceedings of the Royal Society of London. Series A, Containing Papers of a Mathematical and Physical Character. The Royal Society, 1927. Т. 115, № 772. С. 700–721.

2. Hethcote H.W. The Mathematics of Infectious Diseases // SIAM Review. Society for Industrial & Applied Mathematics (SIAM), 2000. Т. 42, № 4. С. 599–653.

3. Lotka A.J. Elements of Physical Biology. Baltimore: Williams; Wilkins Company, 1925. 435 с.

4. Lotka A.J. Contribution to the Theory of Periodic Reaction // The Journal of Physical Chemistry A. 1910. Т. 14, № 3. С. 271–274.

5. Volterra V. Variations and fluctuations of the number of individuals in animal species living together // Journal du Conseil permanent International pour l’ Exploration de la Mer. 1928. Т. 3, № 1. С. 3–51.

6. Вольтерра В. Математическая теория борьбы за существование. Москва: Наука, 1976. 288 с.